

Machine Learning 종합 보고서

Santander Customer Satisfaction · Credit Card Fraud Detection ·
Opiniosis Opinion/Review · Mercari Price Suggestion — 4개 Kaggle
과제에 대한 데이터 전처리, 모델링, 성능 검증 결과 분석

이진분류 · RandomForest

이진분류(불균형) · LightGBM

비지도 군집화 · TF-IDF/SVD

회귀 · LinearRegression

4

분석 대상 데이터셋

1,843,413

4개 데이터셋 총 샘플 수(행)

5

팀 비교 모델(회귀/트리 계열)

2026

작성 연도

담당: 염운호

Kaggle 데이터셋 기반 분석 & 모델링 리포트

목차

0. 초록

1. 서론

- 1.1 연구 배경 및 필요성
- 1.2 과제 목적

2. Santander Customer Satisfaction

- 2.1 프로젝트 개요
 - 2.1.1 입력 피처 구성 · 2.1.2 EDA 요약 통계 · 2.1.3 결측치 현황
 - 2.1.4 Feature 이름 특징 · 2.1.5 동일 값인 Feature · 2.1.6 중복 Feature · 2.1.7 타깃 변수 분포
- 2.2 결과 지표
- 2.3 데이터 전처리 & 클리닝
- 2.4 피처 구조화 & 엔지니어링
- 2.5 모델링 & 하이퍼파라미터 튜닝
- 2.6 성능 검증 결과
 - 2.6.1 파이프라인 요약 · 2.6.2 지표 결과 · 2.6.3 지표 해석 · 2.6.4 모델별 비교/추론

3. Credit Card Fraud Detection

- 3.1 프로젝트 개요 ~ 3.6 성능 검증 결과 (2장과 동일 구성)

4. Opiniosis Opinion / Review

- 4.1 프로젝트 개요 ~ 4.6 성능 검증 결과 (2장과 동일 구성)

5. Mercari Price Suggestion

- 5.1 프로젝트 개요 ~ 5.6 성능 검증 결과 (2장과 동일 구성)

6. 회고

7. 참고문헌

0 초록

4개 Kaggle 과제에 대한 전처리·모델링·성능 검증 결과 요약

본 보고서는 **Santander Customer Satisfaction**(이진분류), **Credit Card Fraud Detection**(이진분류·극단적 불균형), **Opiniosis Opinion/Review**(비지도 군집화·텍스트), **Mercari Price Suggestion**(회귀) 4개의 Kaggle 데이터셋을 대상으로 담당자(염윤호)가 직접 수행한 데이터 전처리, 피처 엔지니어링, 모델링, 성능 검증 결과를 정리한다.

Santander 과제에서는 랜덤포레스트(RandomForestClassifier)로 임계값을 0.05로 조정해 accuracy 0.8074, roc_auc 0.8293, recall 0.6952를 얻었다. Credit Card 과제에서는 LightGBM과 SMOTE 오버샘플링을 결합해 roc_auc 0.9719, recall 0.7959를 얻었다. Opiniosis 과제에서는 TF-IDF와 TruncatedSVD(n_components=5)를 결합한 K-means 군집화가 대리 정답(대상 종류) 기준 ARI 1.000, 안정성 1.000을 달성했다. Mercari 과제에서는 LinearRegression으로 RMSLE 0.4818을 얻었다.

각 과제에서 담당 모델의 결과를 팀 내 다른 모델·다른 팀원의 결과와 비교하여, 특정 지표가 높거나 낮게 나온 원인을 전처리 방식과 모델 특성 관점에서 추론했다. 모든 수치는 각 노트북에 실제로 기록된 실행 결과와 원본 데이터 재검증을 근거로 하며, 추정치는 추정임을 명시한다.

1 서론

연구 배경과 과제 목적

1.1 연구 배경 및 필요성

데이터 기반 의사결정은 경험이나 직관이 아니라 수집된 데이터의 통계적 패턴에 근거해 판단을 내리는 방식이다. 이 방식이 유효하려면 두 가지가 필요하다. 첫째, 데이터의 구조(피처 유형, 결측치, 분포, 클래스 불균형 등)를 정확히 파악해야 한다. 둘째, 문제 유형에 맞는 모델과 평가지표를 선택하고, 그 결과를 올바르게 해석해야 한다.

본 프로젝트는 이진분류(정형 데이터), 이진분류(극단적 불균형 데이터), 비지도 군집화(비정형 텍스트 데이터), 회귀(텍스트·범주형·수치형 혼합 데이터)라는 서로 다른 4가지 문제 유형을 다룬다. 문제 유형이 다르면 전처리 전략과 평가지표도 달라져야 한다. 예를 들어 클래스 비율이 96:4인 데이터에 accuracy만 사용하면 소수 클래스를 전혀 못 맞혀도 높은 점수가 나오는 착시가 발생한다. 이런 함정을 피하기 위해 각 과제마다 지표를 선정한 근거를 먼저 밝히고, 그 지표가 실제로 무엇을 알려주는지 해석하는 절차를 거친다.

1.2 과제 목적

- 데이터 기반으로 의사결정을 하기 위한 머신러닝 학습 및 예측 결과를 확보한다.
- "Santander Customer Satisfaction", "Credit Card Fraud Detection", "Opinions Opinion/Review", "Mercari Price Suggestion" 4개 데이터셋을 대상으로 분석을 수행한다.
- 결과 지표를 수치로 제시하고, 그 수치가 실제 상황에서 어떤 의미를 갖는지 해석하여 과제에 대한 이해를 돕는다.

보고서 구성 안내

2~5장은 각 데이터셋마다 동일한 순서(프로젝트 개요 → 결과 지표 → 전처리 → 피처 엔지니어링 → 모델링 → 성능 검증)로 구성된다. 담당자가 실제로 사용한 모델을 중심으로 서술하며, 다른 모델은 비교 대상으로만 다룬다.

2 Santander Customer Satisfaction

은행 고객의 익명화된 거래 피처로 "불만족(이탈 위험)" 고객을 예측하는 이진분류 문제 — 담당 모델: RandomForest

2.1 프로젝트 개요

고객의 익명화된 369개 피처를 이용해 고객이 불만족(TARGET=1) 상태인지 예측하는 이진분류 문제다. 전체 컬럼은 ID 1개, TARGET 1개, 피처 369개로 총 371개이며, 샘플 수는 76,020건이다.

2.1.1 입력 피처 구성

피처명은 모두 익명화되어 있어 실제 의미(소득, 거래 횟수 등)를 알 수 없고, var3 처럼 코드화된 이름만 제공된다. 다만 접두어 패턴으로 피처군을 구분할 수 있다.

접두어	개수	추정 의미(비공식)
num_*	143	수량/횟수 관련 지표로 추정
ind_*	75	0/1 이진 지표로 추정
saldo_*	71	스페인어 "잔액" 관련 변수로 추정
imp_*	49	스페인어 "금액(importe)" 관련 변수로 추정
delta_*	26	변화율/증감 관련 변수로 추정
var3 / var15 / var38 등	5	단독 변수명, 공식 의미 미공개

주의

접두어의 의미는 컬럼명 어휘에서 유추한 비공식 추정이며, Kaggle 측이 공식적으로 정의를 공개하지 않았다. 본 보고서는 이 해석을 참고용으로만 제시하며 사실로 단정하지 않는다.

2.1.2 EDA 요약 통계

76,020

전체 샘플 수(행)

371

전체 컬럼 수(ID+TARGET+피처 369)

51.27

var38 왜도(skewness)

116건

var3 = -999999 이상치

var38 (추정: 자산가치 관련 금액 변수)은 최솟값 5,163.75, 최댓값 22,034,740, 평균 117,235.8로 왜도 51.27의 극단적 우편향 분포를 가진다. 로그 변환(log1p)을 적용하면 왜도가 0.38까지 낮아져 정규분포에 가까워진다.

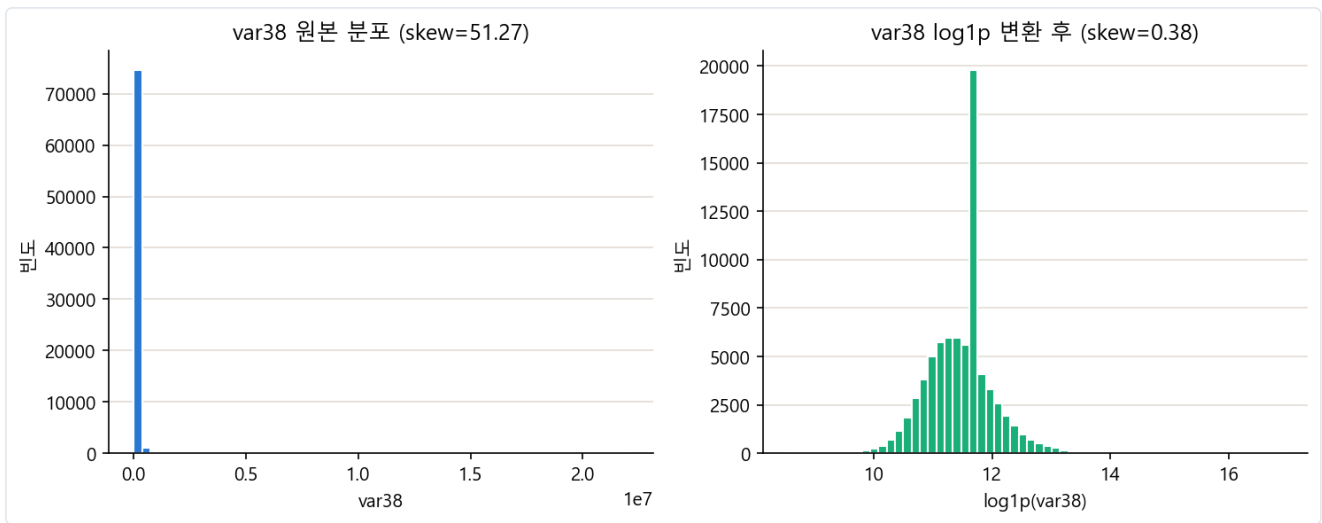


그림 2-1. var38 원본 분포(좌, 왜도 51.27)와 log1p 변환 후 분포(우, 왜도 0.38) — 로그 변환의 잠재적 효과를 보여주는 참고 시각화이며, 2.3~2.4에서 서술하듯 염윤호의 최종 코드에는 이 변환이 실제로 적용되지 않았다.

2.1.3 결측치 현황

원본 데이터를 직접 재검증한 결과 371개 컬럼, 76,020행 전체에서 결측치는 0건이다. 다만 `var3`의 -999999처럼 결측값을 특수 상수로 채워 넣은 것으로 추정되는 값이 있어, "결측치 0건"이 데이터 품질이 완전함을 뜻하지는 않는다.

2.1.4 Feature 이름 특징

피처명이 사람이 읽을 수 없는 코드로 익명화된 것은 은행 고객 데이터의 개인정보 보호 목적으로 추정된다. `_ult1` (최근 1개월), `_ult3` (최근 3개월), `_hace2` / `_hace3` (N개월 전) 같은 접미어 패턴이 관찰되어, 여러 시점의 값을 담은 시계열성 파생 변수군으로 보인다(비공식 추정).

2.1.5 동일 값인 Feature

표준편차가 0인, 즉 전체 행에서 값이 모두 동일한 컬럼이 **34개** 존재한다(`ind_var2_0`, `ind_var27_0`, `num_var46`, `saldo_var41` 등). 이런 컬럼은 클래스를 구분하는 정보를 전혀 담지 못하므로 제거 대상이다.

2.1.6 중복 Feature

원본 369개 피처를 기준으로 값이 완전히 동일한 중복 컬럼은 **62개**다. 염윤호의 노트북은 분산 0 컬럼(34개)을 먼저 제거한 뒤(335개 피처) 중복을 탐색해 **29개**를 추가로 찾았다. 두 수치(62개 vs 29개)가 다른 것은 계산 순서 차이 때문이다. 분산 0인 상수 컬럼끼리는 서로 값이 같으므로 그 자체로 "중복"에 해당하는데, 상수 컬럼을 먼저 제거하면 중복 탐지 대상 풀이 줄어들어 남는 중복 개수도 함께 줄어든다. 둘 다 계산상 오류는 아니다.

2.1.7 타겟 변수 분포 분석

TARGET은 0(만족) 73,012건(96.04%), 1(불만족) 3,008건(3.96%)으로, 약 96:4의 극단적 클래스 불균형이다. 단순히 모든 고객을 "만족"으로 예측해도 accuracy가 약 96%에 이를 수 있는 구조이므로, 2.2에서 다루듯 accuracy만으로는 모델 성능을 판단할 수 없다.

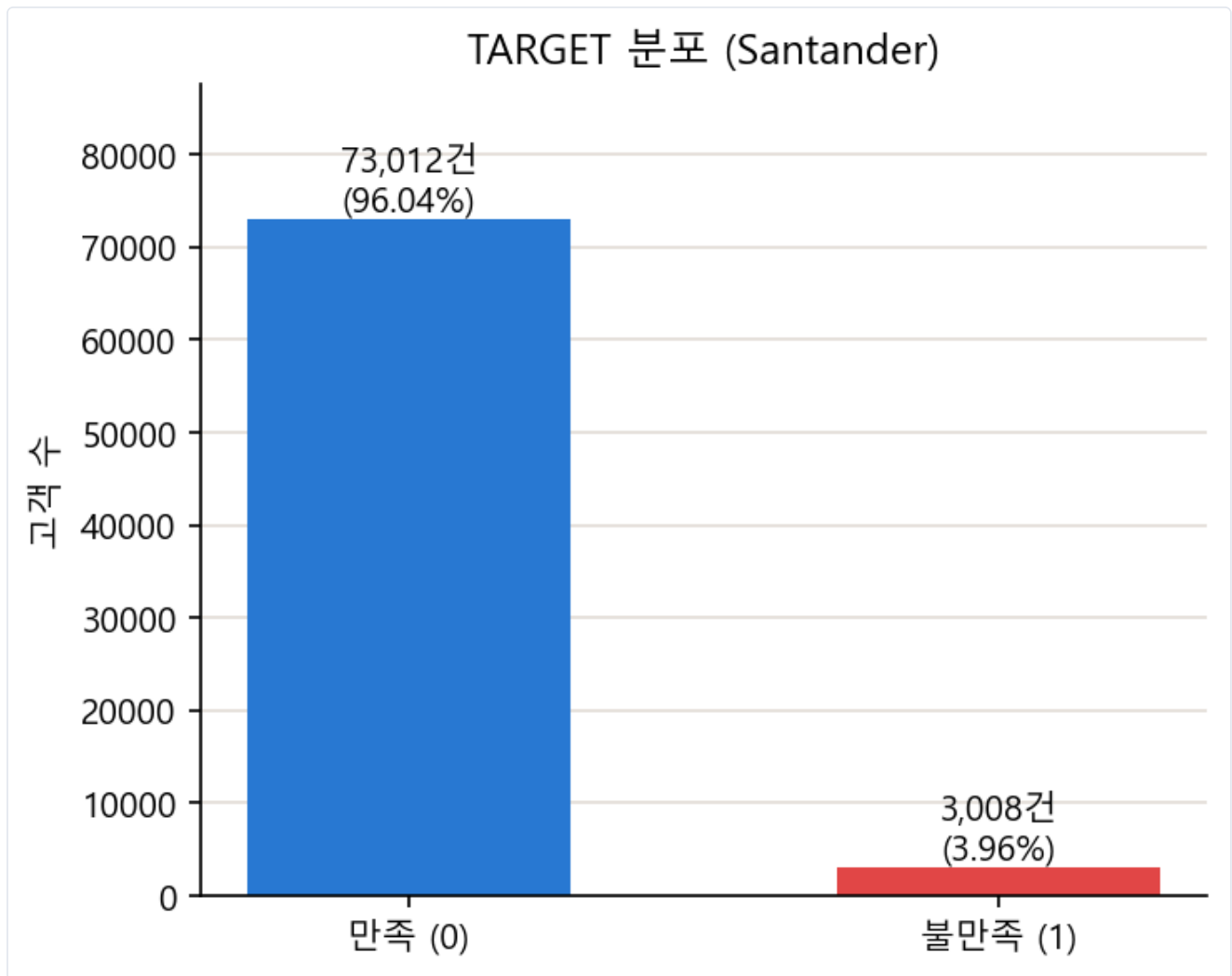


그림 2-2. TARGET 분포 — 만족 73,012건(96.04%) vs 불만족 3,008건(3.96%)

2.2 결과 지표

2.2.1 결과 지표 정의 및 선정 근거

기본 임계값(0.5)에서는 accuracy가 0.9601까지 나오지만 recall은 0.0033에 불과하다. 즉 accuracy가 높아도 불만족 고객을 거의 찾아내지 못하는 모델일 수 있다. 이 때문에 임계값에 의존하지 않고 두 클래스를 얼마나 잘 순위화하는지 보여주는 **ROC-AUC**와, 실제 불만족 고객 중 몇 %를 찾아냈는지를 보여주는 **recall**을 핵심 지표로 병행한다.

왜 accuracy 단독 사용이 위험한가

96:4 불균형 데이터에서는 "전부 만족으로 예측"하는 무전략 모델도 accuracy 96%를 받는다. accuracy는 소수 클래스 탐지 능력을 전혀 반영하지 못하므로, 불균형 데이터에서는 recall-precision-ROC-AUC를 함께 봐야 한다.

2.2.2 결과 지표 해석 계획

임계값을 0.01~0.39 구간에서 스캔하여 정밀도-재현율-F1의 트레이드오프를 확인하고, 재현율을 실용적 수준까지 끌어올리는 임계값을 채택한다. 채택된 임계값에서의 최종 수치를 팀 내 다른 모델·다른 담당자 결과와 비교해 성능 차이의 원인(전처리, 모델 종류, 임계값 조정 여부)을 추론한다(2.6.4).

2.3 데이터 전처리 & 클리닝

염윤호가 실제로 수행한 전처리는 다음 세 가지다.

① var3 이상치 치환 및 ID 제거

```
cust_df["var3"] = cust_df["var3"].replace(-999999, 2)
cust_df.drop("ID", axis=1, inplace=True)
```

-999999는 결측값을 상수로 채운 이상치로 추정되며(116건), 최빈값으로 알려진 2로 치환했다. ID는 예측에 무의미한 식별자이므로 제거했다.

② 분산 0(상수) 컬럼 제거

```
stds = X_features.std()
zero_var_cols = stds[stds == 0].index.tolist()
X_features_clean = X_features.drop(columns=zero_var_cols)
```

34개 컬럼 제거로 피쳐 shape이 (76020, 369) → (76020, 335)로 줄었다.

③ 완전 중복 컬럼 제거

```
key = (v.sum(), v.min(), v.max())
# ... 후보로 좁힌 뒤
if np.array_equal(df[cols[i]].values, df[cols[j]].values):
    dup.add(cols[j])
```

sum/min/max로 후보를 먼저 좁힌 뒤 완전 일치 여부를 검사해, 전체 컬럼 쌍을 무차별 비교하는 것보다 효율적으로 29개의 완전 중복 컬럼을 찾아 제거했다(335 → 306). 희소 컬럼 제거, 스케일링, 결측치 처리는 수행하지 않았다. RandomForest는 트리 기반 모델이라 피쳐 스케일에 영향을 받지 않으므로 스케일링 생략은 합리적인 선택이다.

2.4 피쳐 구조화 & 엔지니어링

엔지니어링 없음

노트북 전체를 확인한 결과, 파생 변수 생성·로그 변환·구간화(binning) 등 피쳐 엔지니어링 코드는 없다. `X_features = cust_df.iloc[:, :-1]`로 TARGET을 분리하는 것 외의 구조 변경 작업은 수행하지 않았다. 그림 2-1의 log1p 변환은 "적용했다면 왜도가 개선됐을 것"이라는 참고용 시각화일 뿐, 실제 모델 학습에는 반영되지 않았다. 다른 팀원(안성준·정영찬)은 var38 로그 변환이나 var15 구간화를 적용했다는 점에서 염윤호의 전처리는 상대적으로 단순하다.

2.5 모델링 & 하이퍼파라미터 튜닝

기본 RandomForest(sklearn 디폴트) 학습 후 교차검증으로 `n_estimators=500`의 효과(ROC-AUC 0.7596±0.0049)를 확인하고, GridSearchCV로 `max_depth`·`min_samples_split`을 탐색했다.


```

params = {
    'n_estimators': [500],
    'max_depth': [28, 32, 40],
    'min_samples_split': [22, 26, 30]
}
grid_cv = GridSearchCV(rf_clf, param_grid=params, scoring='roc_auc', cv=2, n_jobs=-1)
# best: {'max_depth': 28, 'min_samples_split': 30, 'n_estimators': 500}, best AUC 0.8208

```

최종 모델은 `RandomForestClassifier(max_depth=28, min_samples_split=22, n_estimators=500)` 로 학습됐다. 기본 임계값(0.5)에서 accuracy 0.9601, ROC-AUC 0.8293을 얻은 뒤, 임계값을 0.01~0.39 구간에서 스캔해 최종적으로 **임계값 0.05**를 채택했다.

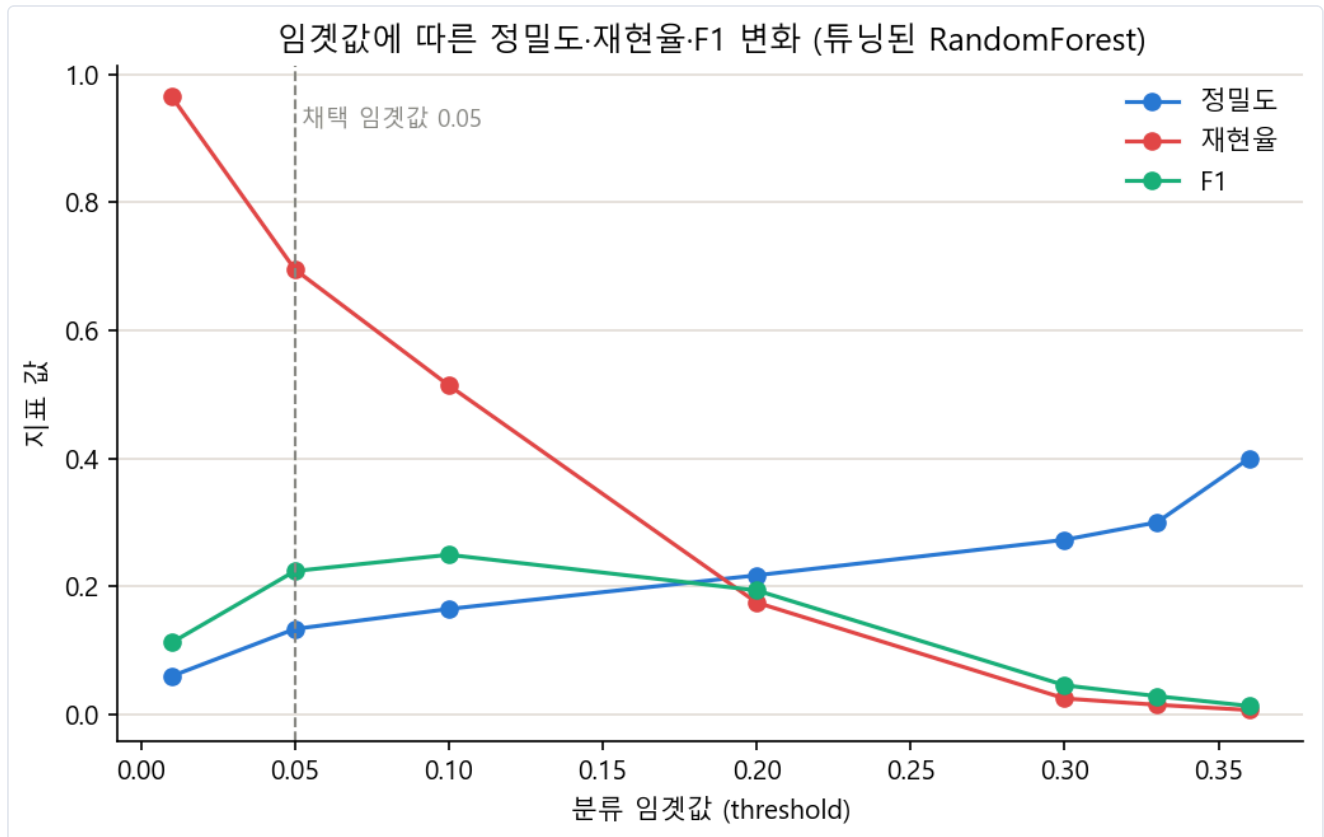


그림 2-3. 임계값에 따른 정밀도·재현율·F1 변화(채택 임계값 0.05, 점선)

임계값이 낮아질수록 재현율은 급격히 높아지고(0.01일 때 0.9654, 0.05일 때 0.6952, 0.1일 때 0.5140) 정밀도는 낮아진다. 0.05는 재현율을 실용적 수준(약 70%)까지 끌어올리면서 정확도 붕괴는 피하는 지점이다. F1만 보면 임계값 0.1(F1 0.2493)이 0.05(F1 0.2238)보다 근소하게 높지만, 이 프로젝트는 F1 최적화보다 재현율(불만족 고객 탐지율)을 우선했다.

확인 필요

GridSearchCV가 찾은 최적값은 `min_samples_split=30`이지만 최종 모델은 이전 탐색 라운드의 값인 `min_samples_split=22`를 사용했다. 노트북에 그 사유가 명시되어 있지 않아 담당자 확인이 필요하다.

2.6 성능 검증 결과

2.6.1 파이프라인 요약 다이어그램



2.6.2 지표 결과

구분	정확도	정밀도	재현율	F1	ROC-AUC
기본 RF(튜닝 전, 임계값 0.05)	0.8204	0.1239	0.5766	0.2040	0.7585
최종 RF(튜닝 후, 기본 임계값 0.5)	0.9601	미확인	미확인	미확인	0.8293
최종 RF(튜닝 후, 채택 임계값 0.05)	0.8074	0.1333	0.6952	0.2238	0.8293

2.6.3 지표 해석

- ROC-AUC 0.8293은 모델이 만족/불만족 고객을 임계값과 무관하게 상당히 잘 순위화한다는 뜻이다(무작위 예측 0.5, 완벽 분류 1.0).
- 재현율 0.6952는 실제 불만족 고객 100명 중 약 70명을 모델이 위험군으로 식별한다는 뜻이다. 은행 입장에서는 이탈 위험 고객 10명 중 약 7명에게 사전 대응(리텐션 캠페인)이 가능해진다.
- 정밀도 0.1333은 모델이 "불만족"이라고 예측한 고객 중 실제 불만족 고객은 약 13%뿐이라는 뜻이다. 나머지 87%는 오탐(실제로는 만족 고객)이다. "놓치는 불만족 고객을 최소화"하는 전략에는 부합하지만, 불필요한 리텐션 비용이 발생할 수 있다.
- 정확도가 0.9601(튜닝 후 기본 임계값)에서 0.8074(채택 임계값)로 낮아진 것은 성능 저하가 아니라, 재현율을 높이기 위해 의도적으로 임계값을 낮춘 트레이드오프의 결과다.

2.6.4 모델별 결과 비교 및 추론

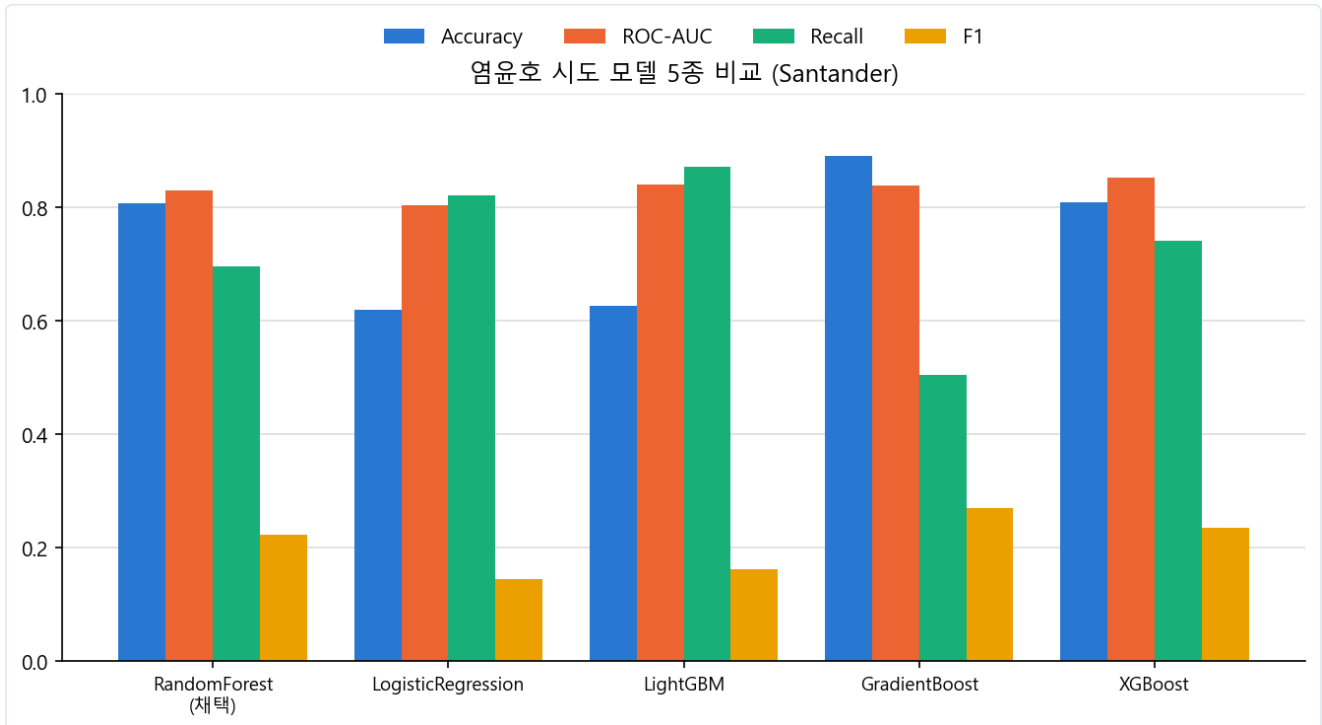


그림 2-4. 염윤호가 시도한 5개 모델 성능 비교(RandomForest 채택)

모델	정확도	ROC-AUC	재현율	F1	임계값
RandomForest(채택)	0.8074	0.8293	0.6952	0.2238	0.05
LogisticRegression	0.6191	0.8032	0.8206	0.1457	미확인
LightGBM	0.6267	0.8408	0.8722	0.1631	미확인
GradientBoost	0.8909	0.8386	0.5041	0.2695	0.1
XGBoost	0.8086	0.8520	0.7409	0.2346	0.05

본인이 시도한 5개 모델 중에서는 XGBoost의 ROC-AUC(0.8520)가 가장 높다. 순수 판별력만 보면 XGBoost가 RandomForest보다 근소하게 우수하다. RandomForest가 최종 채택된 것은 성능 우위 때문이라기보다, 이 프로젝트에서 염윤호에게 배정된 담당 모델이 RandomForest였기 때문으로 보인다. LightGBM은 재현율(0.8722)이 가장 높지만 정확도 (0.6267)와 F1(0.1631)이 가장 낮다. 매우 낮은 임계값으로 재현율을 극대화한 대신 정밀도를 크게 희생한 결과로 추정된다. 반대로 GradientBoost는 F1(0.2695)이 가장 높지만 재현율(0.5041)이 가장 낮다. 임계값도 0.1로 다른 모델보다 높게 설정되어, 상대적으로 "정밀도 우선/보수적 탐지" 전략을 취한 것으로 해석된다.

담당자	모델	정확도	ROC-AUC	재현율	F1	비고
안성준	LogisticRegression	0.7459	0.8116	0.8239	0.1866	임계값 0.04
안성준	RandomForest	0.9509	0.7643	0.5480	0.0813	임계값 조정 미확인
정찬성	LightGBM	0.7796	0.8417	0.7444	0.2198	임계값 0.039
염윤호	RandomForest	0.8074	0.8293	0.6952	0.2238	임계값 0.05
이민석	GradientBoost	0.8938	0.8179	0.4934	0.2690	임계값 0.05

가장 뚜렷한 패턴은 "임계값 조정 여부"가 재현율을 좌우한다는 것이다. 안성준의 RandomForest는 재현율이 0.548에 그치는데, 이는 임계값을 낮게 조정하지 않았기 때문으로 추정된다. 반면 염윤희·정찬성·이민석은 모두 임계값을 0.039~0.1 수준으로 낮춰 재현율을 0.49~0.70대까지 끌어올렸다. 이는 96:4 불균형 데이터에서는 어떤 모델을 쓰는지보다 임계값을 어디에 두는지가 재현율에 더 큰 영향을 준다는 것을 시사한다. ROC-AUC만 보면 안성준의 RandomForest(0.7643)가 염윤희의 RandomForest(0.8293)보다 낮는데, 염윤희는 상대적으로 단순한 전처리에도 GridSearchCV 하이퍼파라미터 튜닝을 거쳤다는 차이가 있다. 다만 안성준의 하이퍼파라미터 설정을 직접 확인하지 못했으므로 이는 추론이며 확정적 결론은 아니다.

확인 필요/미확인 항목

① 튜닝 후 기본 임계값(0.5)에서의 정밀도·재현율·F1 값(노트북 미출력) ② var3를 2로 치환한 구체적 근거 ③ 임계값 0.05를 최종 채택한 명시적 사유(F1은 임계값 0.1이 근소하게 더 높음) ④ 안성준·정찬성·이민석의 임계값 조정 여부 및 상세 전처리 방식.

3 Credit Card Fraud Detection

PCA로 익명화된 거래 변수로 사기(fraud) 거래를 탐지하는 극단적 불균형 이진분류 문제 — 담당 모델: LightGBM + SMOTE

3.1 프로젝트 개요

신용카드 거래 내역으로 사기 거래(Class=1) 여부를 예측하는 이진분류 문제다. 전체 284,807건, 31개 컬럼 (Time, V1~V28, Amount, Class)으로 구성되며, 사기 비율이 0.173%에 불과한 Santander보다도 훨씬 극단적인 불균형 데이터다.

3.1.1 입력 피처 구성

V1~V28은 원본 거래 정보(가맹점, 위치, 결제 수단 등으로 추정)를 PCA로 변환한 연속형 변수다. 카드사 고객·가맹점의 민감한 금융 개인정보이기 때문에 데이터 제공자가 PCA로 익명화해 공개했고, 그 결과 V1~V28의 실제 의미는 역추적이 불가능하다. 익명화되지 않은 피처는 Time (첫 거래 이후 경과 초)과 Amount (거래 금액) 두 개뿐이다.

3.1.2 EDA 요약 통계

284,807

전체 샘플 수(행)

31

전체 컬럼 수

0.173%

사기 거래 비율

16.98

Amount 왜도(skewness)

V1~V28은 모두 평균이 0에 매우 가깝고(PCA 변환 특성) 표준편차는 0.33~1.96으로 변수마다 다르다. Amount는 평균 88.35, 표준편차 250.12, 최댓값 25,691.16으로 오른쪽 꼬리가 매우 긴 분포다. Time은 최댓값 172,792초로 약 48시간 구간의 거래를 담고 있다.

3.1.3 결측치 현황

원본 데이터를 직접 재검증한 결과 결측치는 0건이다.

3.1.4 Feature 이름 특징

V + 일련번호 형태는 PCA 변환 후 주성분(Principal Component) 순서를 그대로 붙인 익명화 네이밍 패턴이다. 이름만으로는 원본 변수를 전혀 알 수 없고, Time · Amount · Class 만 의미가 드러난다.

3.1.5 동일 값인 Feature

nunique()로 직접 확인한 결과 모든 컬럼이 2개 이상의 고유값을 가진다. 상수(단일 값) 컬럼은 없다.

3.1.6 중복 Feature

컬럼을 전치해 완전 동일 여부를 확인한 결과 중복되는 피처(컬럼) 쌍은 없다. 참고로 행 기준 완전 중복은 1,081건 존재하지만, 이 노트북에서는 행 중복 제거를 전처리에 포함하지 않았다.

3.1.7 타깃 변수 분포 분석

Class 분포는 정상(0) 284,315건(99.827%), 사기(1) 492건(0.173%)이다. 사기 거래가 전체의 0.173%에 불과해, accuracy 기반 평가가 완전히 무의미해지는 전형적인 사례다.

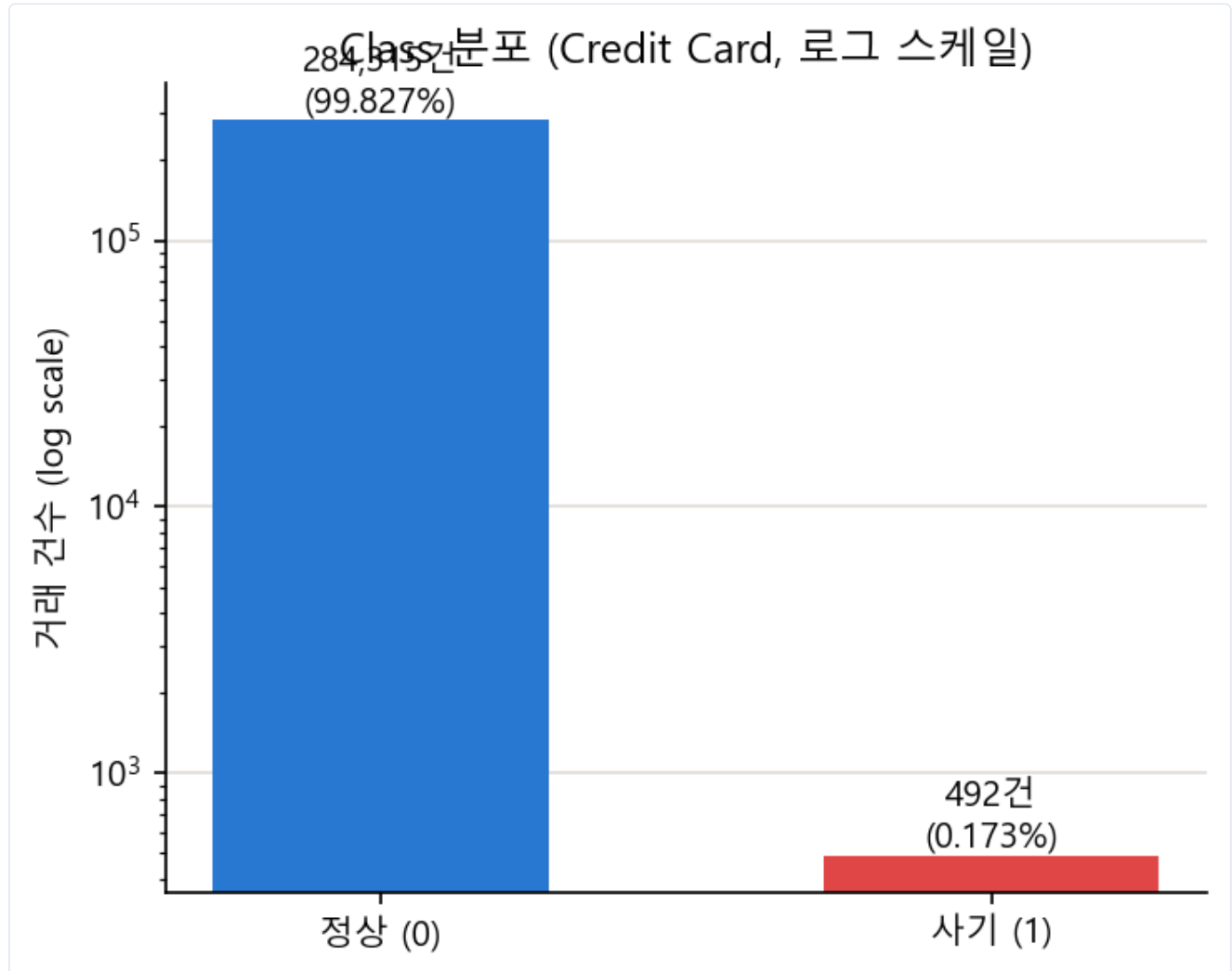


그림 3-1. Class 분포(로그 스케일) — 사기 거래는 전체의 0.173%에 불과하다.

3.2 결과 지표

3.2.1 결과 지표 정의 및 선정 근거

accuracy의 함정

이 데이터에서 모든 거래를 "정상"이라고만 예측해도 accuracy는 $284,315/284,807 \approx 99.827\%$ 가 나온다. 사기를 단 한 건도 잡지 못해도 accuracy가 99%대로 계산되는 함정이 존재한다.

그래서 recall·precision·F1·AUPRC(average_precision)를 함께 본다. recall은 실제 사기 중 몇 %를 잡아냈는지, precision은 사기로 예측한 것 중 실제 사기 비율을 보여준다. AUPRC는 불균형 데이터에서 ROC-AUC보다 클래스 불균형에 덜 민감하고 임계값 전반의 정밀도-재현율 트레이드오프를 종합적으로 보여줘, 하이퍼파라미터 튜닝의 기준 지표로도 채택됐다(노트북 주석: "불균형 데이터라 accuracy 대신 AUPRC 사용").

3.2.2 결과 지표 해석 계획

혼동행렬(confusion matrix)로부터 accuracy/precision/recall/F1/AUC/AUPRC를 `get_clf_eval()` 함수로 일괄 산출한다. 모델·샘플링 기법 간 비교는 accuracy 단독이 아니라 recall·precision·F1·AUPRC를 함께 놓고 트레이드오프를 판단한다.

3.3 데이터 전처리 & 클리닝

```
def get_preprocessed_df(df=None):
    df_copy = df.copy()
    amount_n = np.log1p(df_copy['Amount'])
    df_copy.insert(0, 'Amount_Scaled', amount_n)
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
    return df_copy
```

`Time` 은 모델 학습에 불필요한 시간 정보로 판단해 제거했고, `Amount` 는 `log1p` 변환으로 치우친 금액 분포를 완화했다(왜도 16.98 → 0.16).

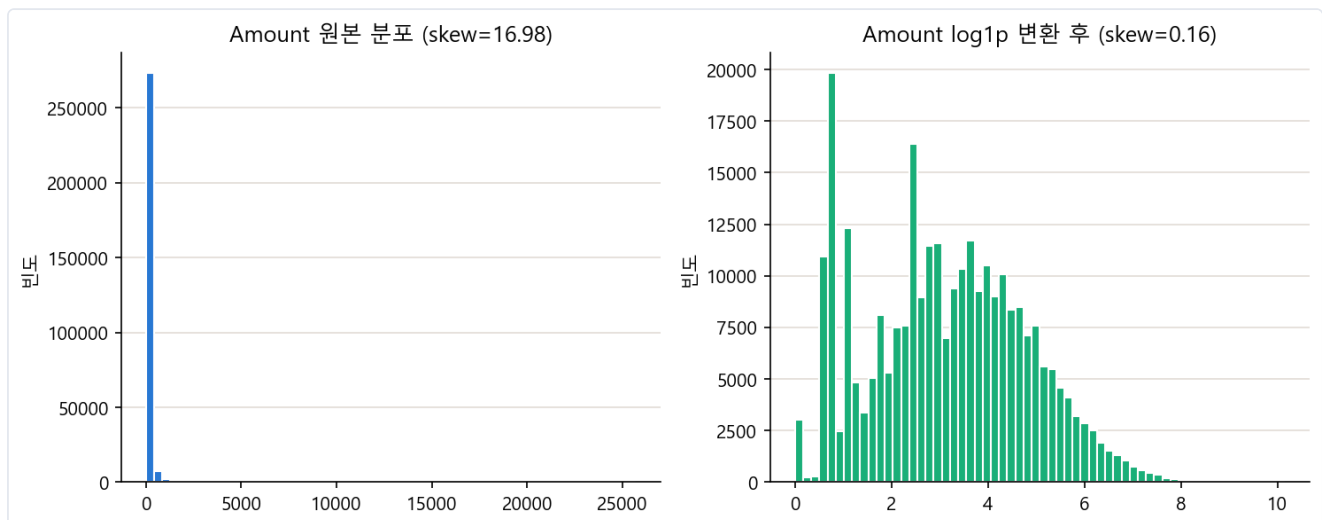


그림 3-2. Amount 원본 분포(좌, 왜도 16.98)와 log1p 변환 후 분포(우, 왜도 0.16)

```
def get_outlier(df, column, weight=1.5):
    fraud = df[df['Class']==1][column]
    q25, q75 = np.percentile(fraud.values, 25), np.percentile(fraud.values, 75)
    iqr = q75 - q25
    lowest, highest = q25 - iqr*weight, q75 + iqr*weight
    return fraud[(fraud < lowest) | (fraud > highest)].index
```

사기 클래스와 상관관계가 큰 `v14` 기준으로 IQR(사분위범위) 1.5배 규칙을 적용해 이상치를 제거한다. train/test 분할 이후 각 세트에 독립적으로 적용해 데이터 누수(data leakage)를 방지한다.

3.4 피처 구조화 & 엔지니어링

SMOTE 오버샘플링을 피처 엔지니어링 관점에서 보면, 소수 클래스(사기) 샘플의 특징 공간에서 최근접 이웃 간 보간(interpolation)으로 새로운 합성 샘플을 만들어 학습 데이터의 클래스 분포 자체를 재구성하는 기법이다.

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train_over, y_train_over = smote.fit_resample(X_train, y_train)
```

적용 결과 학습 데이터가 (199,362, 29) → (398,040, 29)로 늘어나 정상 199,020건 : 사기 199,020건의 1:1 균형을 이룬다. 테스트 데이터에는 SMOTE를 적용하지 않고 원본 분포를 그대로 유지해, 실제 서비스 환경과 동일한 조건에서 성능을 검증한다.

3.5 모델링 & 하이퍼파라미터 튜닝

```
lgbm_clf = LGBMClassifier(n_estimators=1000, num_leaves=64, n_jobs=-1, boost_from_average=False)
```

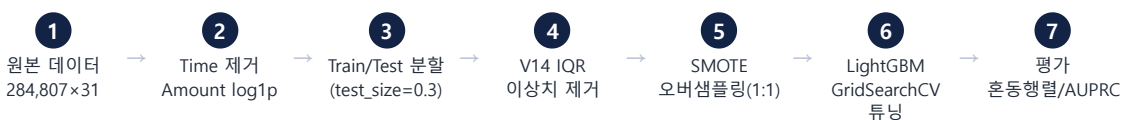
튜닝은 SMOTE 파이프라인에 대해서만 GridSearchCV로 수행했다. 교차검증 폴드 내부에서만 SMOTE를 적용해 (imblearn.pipeline.Pipeline) 검증 폴드로의 데이터 누수를 방지한다.

```
smote_pipeline = ImbPipeline([('sampler', SMOTE(random_state=42)),
                               ('classifier', LGBMClassifier(n_estimators=1000, n_jobs=1, boost_from_average=False))])
param_grid = {'classifier__num_leaves':[31,64], 'classifier__max_depth':[-1,7], 'classifier__learning_rate':[0.05,0.1]}
lgbm_smote_grid = GridSearchCV(smote_pipeline, param_grid, cv=StratifiedKFold(5), scoring='average_precision', n_jobs=-1)
# Best Params: {'learning_rate': 0.05, 'max_depth': -1, 'num_leaves': 64}, Best CV AUPRC: 0.8673
```

5-fold × 2×2×2=8개 조합의 그리드서치로 최적 파라미터를 찾은 뒤, 이 값으로 최종 모델을 재학습했다.

3.6 성능 검증 결과

3.6.1 파이프라인 요약 다이어그램



3.6.2 지표 결과

지표	값
혼동행렬	[[85278, 17], [30, 117]]
Accuracy	0.9994
Precision	0.8731
Recall	0.7959
F1-Score	0.8327
ROC-AUC	0.9720
AUPRC	0.8342

3.6.3 지표 해석

- Recall 0.7959는 실제 사기 거래 147건 중 117건을 모델이 탐지하고(약 80%), 30건은 놓쳤다는 뜻이다.
- Precision 0.8731은 모델이 "사기"라고 예측한 134건 중 117건(약 87%)이 실제 사기이고, 17건은 정상 거래를 잘못 판단한 오탐이라는 뜻이다.
- AUPRC 0.8342는 임계값을 바꿔가며 본 정밀도-재현율 트레이드오프를 종합했을 때도 0.83 수준의 높은 판별력을 유지한다는 뜻으로, 극단적 불균형 데이터에서 우수한 성능으로 평가된다.
- 정상 거래 85,295건 중 17건만 사기로 오탐(오탐율 약 0.02%)해, 정상 고객 경험에 미치는 부정적 영향은 매우 낮다.

3.6.4 모델별 결과 비교 및 추론

(a) 모델 5종 비교(SMOTE 적용)

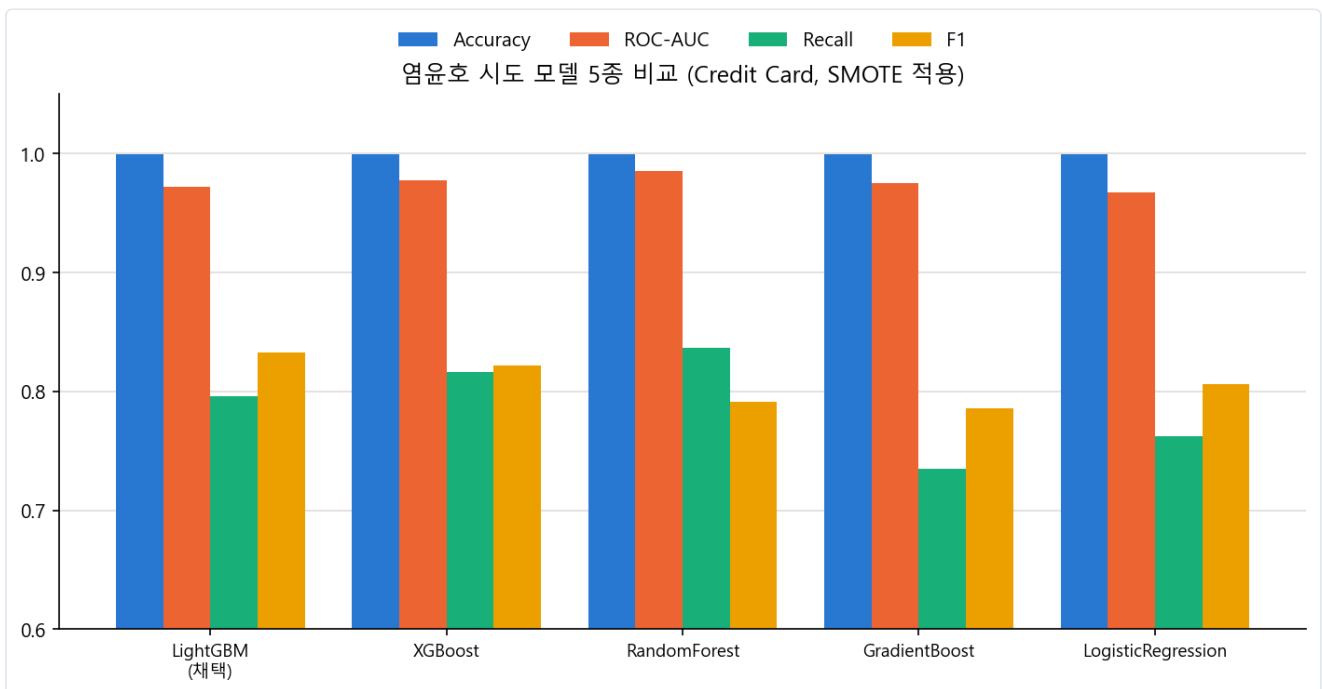


그림 3-3. 염윤호가 시도한 5개 모델 성능 비교(SMOTE 적용, LightGBM 채택)

모델	Accuracy	ROC-AUC	Recall	F1
LightGBM(채택)	0.9994	0.9719	0.7959	0.8327
XGBoost	0.9993	0.9771	0.8163	0.8219
RandomForest(threshold 0.305)	0.9994	0.9854	0.8367	0.7910
GradientBoost	0.9993	0.9749	0.7346	0.7854
LogisticRegression	0.9993	0.9675	0.7619	0.8057

ROC-AUC는 임계값 조정을 포함한 RandomForest(0.9854)가 가장 높지만, 튜닝 없이 기본 F1로 비교하면 LightGBM이 가장 균형 잡힌 F1(0.8327)을 보인다. XGBoost는 recall이 가장 높아(0.8163) 사기를 더 많이 잡아내지만 precision과의 트레이드오프가 있다. GradientBoost는 recall이 가장 낮아(0.7346) 사기 탐지 관점에서는 상대적으로 약하다.

(b) 샘플링 기법 비교 — LightGBM 고정, 튜닝 전(기본 파라미터) 기준

비교 조건

아래 3개 수치는 모두 같은 기본 파라미터(LGBMClassifier(n_estimators=1000, num_leaves=64, n_jobs=-1, boost_from_average=False)), GridSearchCV 튜닝 이전 결과다. SMOTE만 이후 별도로 튜닝했으므로(3.6.2의 튜닝 후 수치), 샘플링 기법 간 공정 비교를 위해 여기서는 튜닝 전 수치만 사용한다.

샘플링 기법	Accuracy	Precision	Recall	F1	ROC-AUC	AUPRC
언더샘플링	0.9695	0.0478	0.8844	0.0906	0.9777	0.6689
SMOTE(튜닝 전)	0.9994	0.8731	0.7959	0.8327	0.9697	0.8274
SMOTE-ENN	0.9992	0.7453	0.8163	0.7792	0.9739	0.8059

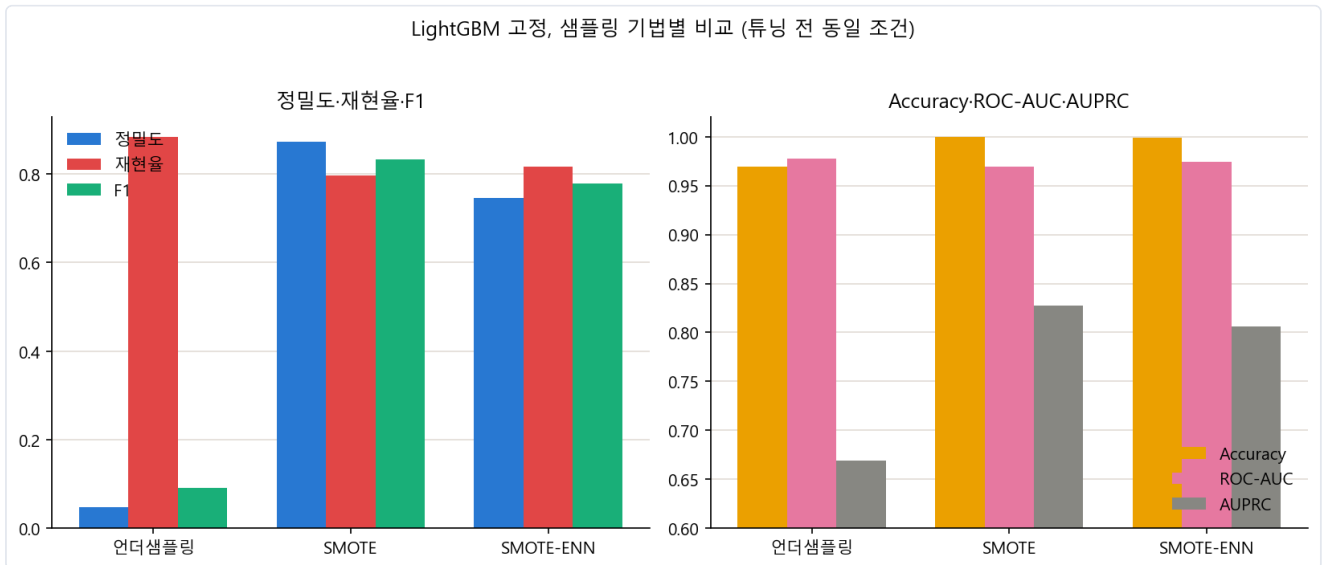


그림 3-4. LightGBM 고정, 샘플링 기법별 비교(튜닝 전 동일 조건)

언더샘플링은 재현율(0.8844)은 가장 높지만 정밀도가 0.0478로 극도로 낮다. 사기라고 예측한 거래 20건 중 1건만 실제 사기라는 뜻으로, 오탐이 너무 많아 실무 적용이 어렵다. 이는 언더샘플링이 정상 데이터를 342건까지 줄여 학습 데이터 절대량이 크게 줄어든 데서 기인한다. SMOTE는 정밀도(0.8731)와 F1(0.8327)이 가장 우수해 균형 잡힌 성능을 보인다. SMOTE-ENN은 SMOTE보다 재현율은 약간 높지만(0.8163) 정밀도가 낮아져(0.7453) F1이 오히려 하락한다(0.7792 < 0.8327). ENN이 경계 근처의 다수 클래스뿐 아니라 일부 유용한 샘플까지 함께 정리(clean)하면서, 정밀도 손실이 재현율 이득보다 크게 나타난 것으로 추정된다. 종합적으로 이 데이터·모델 조합에서는 SMOTE가 세 기법 중 F1 기준 최선이며, 팀이 SMOTE를 최종 채택한 근거와 일치한다.

확인 필요/미확인 항목

- ① Class와 상관관계가 가장 큰 변수는 직접 재계산 결과 V17(-0.3265)이 V14(-0.3025)보다 근소하게 크나, 노트북은 이상치 제거 기준으로 V14를 사용(의도된 선택일 수 있음)
- ② train/test 분할 후 각각 실제 제거된 이상치 건수는 노트북에 별도 출력이 없어 미확인
- ③ SMOTE-SMOTE-ENN 튜닝 전 수치는 노트북 주석으로만 남아 있으나, 언더샘플링 라이브 실행값 및 저장된 차트 수치와 정확히 일치해 신뢰도는 높음
- ④ 행 기준 완전 중복 1,081건은 전처리에서 제거되지 않음.

4 Opiniosis Opinion / Review

전자제품·호텔·자동차 리뷰 51건을 벡터화해 군집화하고 감성을 분석하는 비지도학습 문제

4.1 프로젝트 개요

전자제품·호텔·자동차에 대한 리뷰 문장을 주제별로 모아둔 텍스트 데이터다. 지도학습과 달리 정답 레이블이 없는 **비지도 군집화** 문제이며, 문서 51개를 벡터화한 뒤 군집화하고 감성분석(긍정/부정)까지 수행한다. "타깃 변수", "accuracy" 같은 지도학습 개념은 그대로 적용할 수 없어, 이후 절에서 대리 정답(proxy label)과 외부평가지표(ARI/NMI)로 재해석한다.

4.1.1 입력 피처 구성

`data/topics/` 폴더 안의 파일 51개가 원본 데이터다. 파일 하나가 "하나의 주제(aspect)에 대한, 하나의 대상(제품/호텔/자동차)에 대한 리뷰 문장 모음"이다. 예: `rooms_bestwestern_hotel_sfo.txt.data` → 주제 `rooms` + 대상 `bestwestern_hotel_sfo`. "피처"는 표 형태 컬럼이 아니라 문서 51개 각각의 원문 텍스트이며, 벡터화 단계(4.4)에서 비로소 4,610개의 단어(n-gram) 피처로 구조화된다.

4.1.2 EDA 요약 통계

51

문서(파일) 수

7,086

총 문장 수

138.9

문서당 평균 문장 수

4,610

벡터화 후 피처(단어) 차원

문서당 문장 수는 최소 50~최대 575건으로 편차가 크다. VADER 감성분석에서 집계한 문장 수 합계 ($4,420+1,521+1,145=7,086$)가 원본 직접 계산치와 정확히 일치해 상호 검증됐다.

4.1.3 결측치 현황

표 형태의 결측치 개념은 적용되지 않는다. 51개 파일 모두 정상적으로 읽혀 빈 문서는 없다. 대신 "결측"에 대응하는 것은 문서별 분량 편차(50~575문장)이며, 짧은 문서는 벡터화 시 정보량이 상대적으로 적을 수 있다.

4.1.4 Feature 이름 특징

파일명 규칙은 `{주제}_{대상}` 형태다. 대리 정답을 만들기 위해 접미사 매칭을 적용하는 과정에서 표기 불일치 3건을 확인해 정규화했다.

불일치 사례	정규화
<code>netbook_1005ha</code> ↔ <code>asus_netbook_1005ha</code>	동일 기기(넷북)로 통합
<code>swissotel_chicago</code> ↔ <code>swissotel_hotel_chicago</code>	동일 호텔로 통합
<code>room</code> ↔ <code>rooms</code>	동일 주제(객실)로 통합

4.1.5 동일 값인 Feature

텍스트 원본에는 해당 개념이 없다. 벡터화 이후 관점에서는 "거의 모든 문서에 고르게 등장해 변별력이 없는 단어"가 이에 대응하며, `max_df=0.85` 설정이 이런 단어를 사전에 제거한다.

4.1.6 중복 Feature

51개 파일의 해시를 비교한 결과 중복 파일은 없다. 벡터 차원에서 완전히 동일한 값을 갖는 컬럼이 있는지는 확인하지 않았다(미확인).

4.1.7 "타깃 변수" 분포 분석 (대리 정답으로 대체)

사전 정의된 타깃은 없으므로, 파일명에서 추출한 대리 정답(proxy label) 두 종류로 대체한다.

- **대상 종류(3클래스):** electronics 25개 / hotel 16개 / car 10개
- **주제(35클래스):** battery-life, mileage, rooms, service 등 — 문서 51개 대비 클래스가 너무 많아(평균 1.46개/클래스) 정량 평가에는 한계가 있다.

4.2 결과 지표

4.2.1 결과 지표 정의 및 선정 근거

실루엣 계수를 차원 간 비교의 주력으로 쓰지 않는 이유

고차원 희소 공간(TF-IDF 4,610차원)에서는 거리 집중(distance concentration) 현상 때문에 실루엣이 구조적으로 낮게 나온다. 저차원으로 축소하면 그 자체만으로 실루엣이 유리해지는 함정이 있어, 서로 다른 차원의 두 표현을 비교할 때는 실루엣 대신 대리 정답 기반 외부평가지표(ARI·NMI)를 주력으로 쓴다.

- **실루엣 계수:** 같은 차원 안에서 조합 간 우열(4.5절 5가지 조합)을 비교하는 데 사용.
- **ARI(Adjusted Rand Index):** 군집 결과와 대리 정답 사이의 일치도(우연 수준 보정). 1에 가까울수록 일치.
- **NMI(Normalized Mutual Information):** 군집 결과와 대리 정답 사이의 공유 정보량(정규화).
- **군집 안정성:** 30개 랜덤시드 반복 결과 간 쌍별 ARI 평균. 초기화 민감도를 나타낸다.

4.2.2 결과 지표 해석 계획

ARI/NMI는 "대상 종류(3클래스)" 기준일 때만 실질적 판단력을 갖는다고 보고, "주제(35클래스)" 기준은 참고용으로만 본다(클래스 수 35 > 군집 수 3이라 원천적으로 완전한 분리가 불가능). 감성분석(VADER) 결과는 군집화의 정답이 아니라 군집별 리뷰 성향을 부가 설명하는 용도로만 사용한다.

4.3 데이터 전처리 & 클리닝

```
remove_punct_dict = dict((ord(punct), None) for punct in string.punctuation)
lemmar = WordNetLemmatizer()

def LemNormalize(text):
    return [lemmar.lemmatize(t) for t in nltk.word_tokenize(text.lower().translate(remove_punct_dict))]
```

소문자 변환 → 구두점 제거 → 단어 토큰화 → 표제어(lemma) 변환 순서로 처리하며, 이 함수를 벡터화 단계의 `tokenizer` 인자로 그대로 넘긴다. 불용어는 별도 리스트 없이 `stop_words='english'` (sklearn 내장)로 처리했다. 감성

분석용으로는 문서 단위가 아닌 문장 단위로 별도 로딩해, 문서 병합 시 문장 수 편차로 인한 쓸림을 피했다.

4.4 피처 구조화 & 엔지니어링

```
tfidf_vect = TfidfVectorizer(tokenizer=LemNormalize, stop_words='english',
                             ngram_range=(1,2), min_df=0.05, max_df=0.85)
# → (51, 4610)
count_vect = CountVectorizer(tokenizer=LemNormalize, stop_words='english',
                              ngram_range=(1,2), min_df=0.05, max_df=0.85)
# → (51, 4610), TF-IDF와 동일 파라미터로 벡터화 "방식"만 다르게 비교
count_feature_vect_norm = Normalizer(norm='l2').fit_transform(count_feature_vect)
```

`ngram_range=(1,2)` 로 "gas mileage", "battery life"처럼 두 단어가 붙어야 의미가 분명한 표현을 포착한다. `min_df=0.05 / max_df=0.85` 로 너무 희귀하거나 너무 흔한 단어를 제거한다. Count 벡터는 TF-IDF와 달리 자체 정규화가 없어 문서 길이가 벡터 크기를 지배하는 문제가 있는데, 이를 검증하기 위해 L2 정규화 버전을 추가로 만들었다 (4.6.3의 핵심 근거로 사용).

4.5 모델링 & 하이퍼파라미터 튜닝

```
km_cluster = KMeans(n_clusters=3, max_iter=10000, random_state=0)
```

처음 `n_clusters=5` 로 탐색한 뒤 도메인 수(전자제품/호텔/자동차 3종)에 맞춰 `n_clusters=3` 으로 재수행했다. 계획서(`opinion_review.md`)의 벡터화 2중×군집화 2중(4가지) 비교에 Count 벡터 L2 정규화 버전을 더해 **5가지 조합**을 비교했다.

```
# DBSCAN: 문서 51개로 적어 min_samples=3 고정, eps는 실루엣 최대화 기준 자동 탐색(코사인 거리 5~55백분위)
best_eps, _ = search_dbSCAN(X, eps_grid, min_samples=3) # TF-IDF: 0.7199 / Count: 0.7983
```

감성분석은 `nltk.sentiment.SentimentIntensityAnalyzer` (VADER)로 문장 단위 `compound` 점수를 계산해 ≥ 0.05 는 긍정, ≤ -0.05 는 부정, 그 외는 중립으로 분류한 뒤 문서 단위로 집계했다.

4.6 성능 검증 결과

4.6.1 파이프라인 요약 다이어그램



4.6.2 지표 결과

① 벡터화×군집화 5가지 조합

조합	군집 수	노이즈 비율	실루엣 점수
TF-IDF + KMeans	3	0.000	0.0573

조합	군집 수	노이즈 비율	실루엣 점수
Count + KMeans	3	0.000	0.5489
Count(L2정규화) + KMeans	3	0.000	0.0625
TF-IDF + DBSCAN	4	0.275	0.1249
Count + DBSCAN	2	0.118	0.2194

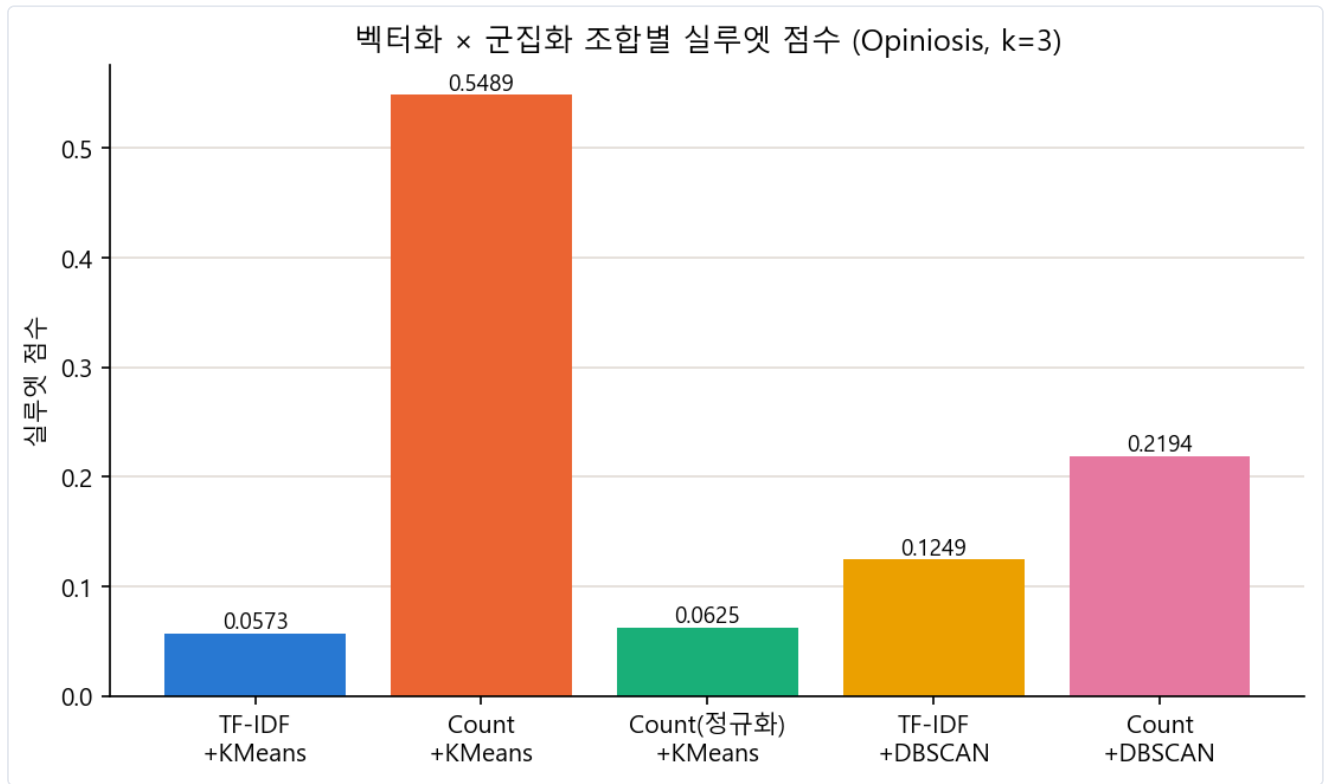


그림 4-1. 벡터화×군집화 조합별 실루엣 점수 비교 — Count+KMeans만 유독 높게 나타난다.

② TF-IDF vs TruncatedSVD 파이프라인 비교(30시드 평균, n_clusters=3 고정, 대상 종류 기준 대리 정답)

파이프라인	n_components	ARI	NMI	안정성
A(TF-IDF, baseline)	—	0.9966	0.9954	0.9932
B(SVD)	2	0.7419	0.7550	1.0000
B(SVD, 최적)	5	1.0000	1.0000	1.0000
B(SVD)	10	0.9219	0.9494	0.8609
B(SVD)	20	0.9682	0.9716	0.9379
B(SVD)	30	0.9737	0.9763	0.9484
B(SVD)	40	0.9913	0.9892	0.9827

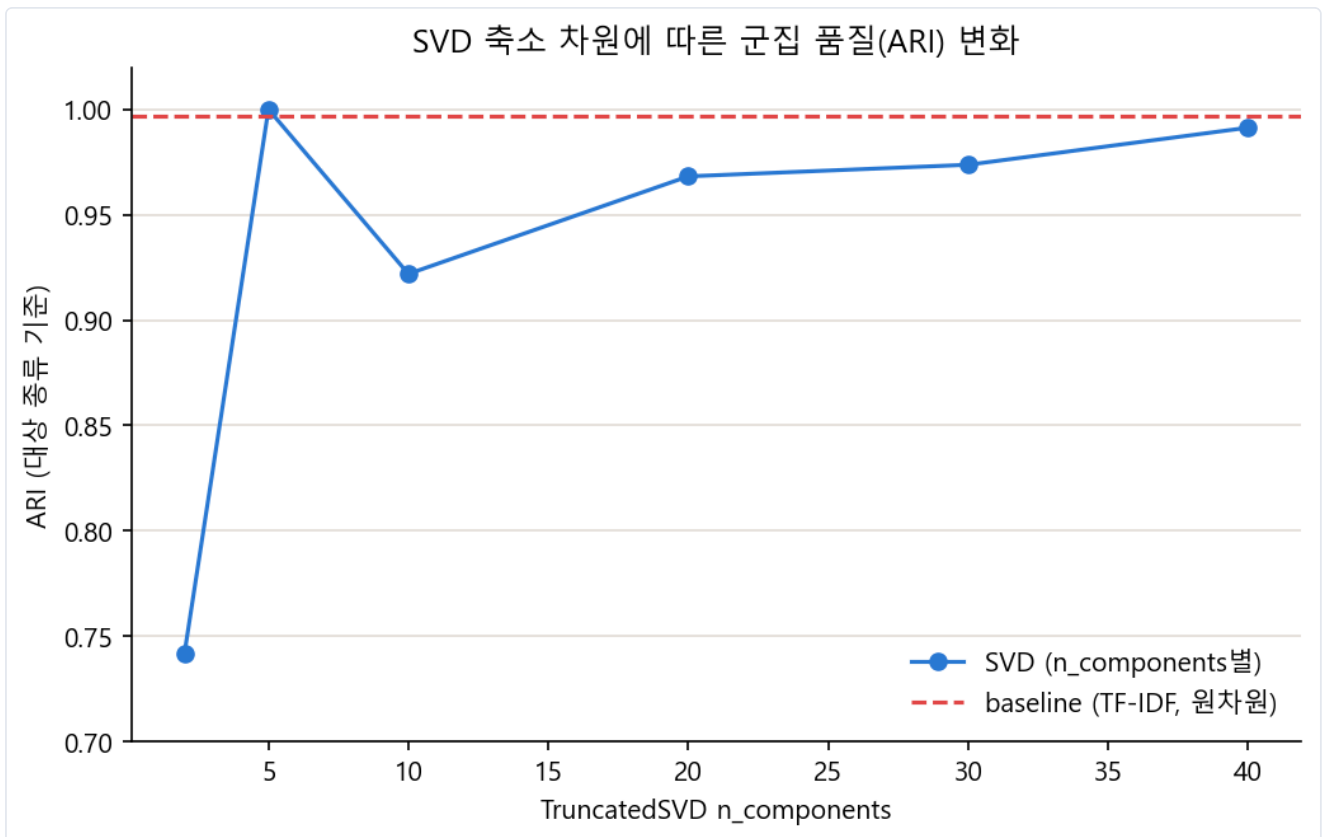


그림 4-2. TruncatedSVD 축소 차원 수에 따른 군집 품질(ARI, 대상 종류 기준) 변화 — n_components=5에서 정점을 찍고 10에서 하락했다가 다시 완만히 회복한다.

③ 감성분석 결과(전체 문장 7,086건 기준)

62.4% 긍정	21.5% 중립	16.2% 부정
--------------------	--------------------	--------------------

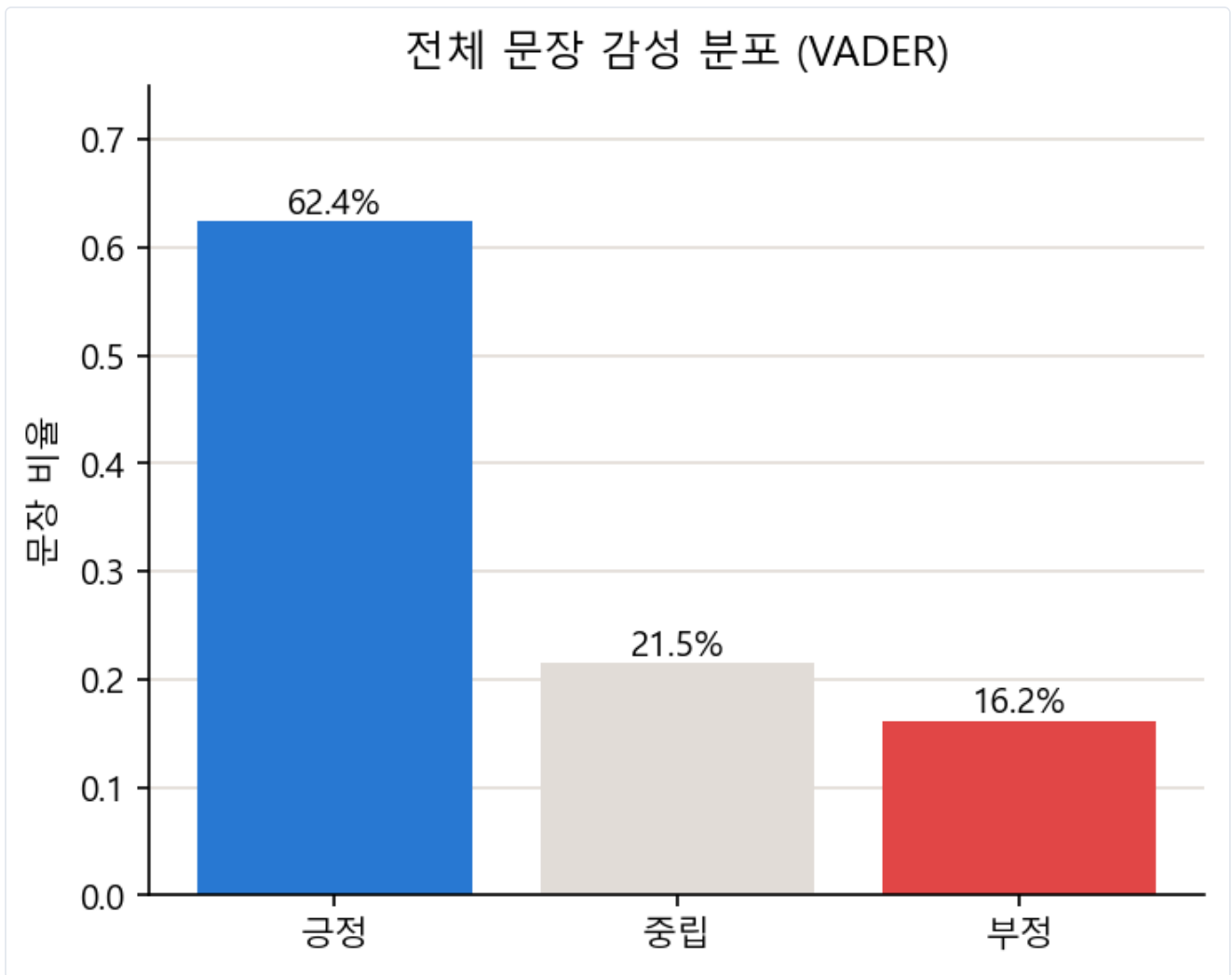


그림 4-3. 전체 리뷰 문장 감성 분포(VADER)

4.6.3 지표 해석

- TF-IDF+KMeans의 실루엣(0.0573)이 낮은 것은 고차원 거리 집중 현상 때문이며, 같은 벡터로 계산한 ARI가 0.9966(거의 완벽)이라는 점이 이를 뒷받침한다. 실루엣만 보면 나쁜 군집처럼 보이지만 실제로는 도메인을 거의 완벽히 분리한 군집이다.
- Count+KMeans의 실루엣(0.5489)이 유독 높은 것은 Count 벡터가 정규화 없이 문서 길이(총 단어 수)에 지배되기 때문으로 추정된다. 같은 벡터에 L2 정규화만 추가한 조합의 실루엣이 0.0625로 급락한다는 점이 이를 뒷받침한다.
- SVD는 $n_components=5$ 일 때만 baseline보다 미세하게 더 좋고(ARI/안정성 1.0), 그 이상 늘리면 오히려 낮아진다. 주제 기준 ARI(0.05 미만)는 $n_clusters=3$ 으로는 35개 세부 주제를 애초에 구분할 수 없기 때문이지 SVD와는 무관하다.
- 전체 리뷰 문장의 62.4%가 긍정으로, 이 데이터셋의 리뷰어들은 전반적으로 만족도가 높은 편이다.

4.6.4 모델별(조합별) 결과 비교 및 추론

핵심 발견 — 실루엣과 ARI가 반대 결론을 준다

실루엣은 벡터 공간의 기하학적 분리도를, ARI는 실제 의미(도메인)와의 일치도를 본다. 이 데이터셋에서는 두 지표가 상반된 인상을 준다 — TF-IDF+KMeans는 실루엣 최하위(0.0573)지만 ARI 최상위(0.9966)다.

Count+KMeans가 실루엣만 높은 이유: 정규화만 켜다 켜다 했을 뿐인데 실루엣이 0.5489 → 0.0625로 9배 가까이 떨어졌다. Count+KMeans의 "좋아 보이는" 결과가 실제로는 '무엇에 대한 리뷰인가'가 아니라 '리뷰가 얼마나 긴가'를 기준으로 군집이 갈린 결과일 가능성이 크다. TF-IDF가 자체적으로 정규화를 내장하고 있어 이런 함정을 피할 수 있다.

DBSCAN vs K-means: DBSCAN은 애매한 문서를 노이즈로 분리할 수 있다는 장점이 있지만, eps를 실루엣 최대화 기준으로 자동 탐색했기 때문에 군집 수가 도메인 수(3)와 다르게 나왔고(TF-IDF 4개, Count 2개) 노이즈 비율도 11.8~27.5%로 낮지 않다. 도메인 수를 이미 알고 있는 이 문제에서는 군집 수를 고정할 수 있는 K-means가 해석에 더 적합하다.

종합: 이 데이터셋에서는 (1) TF-IDF가 Count보다 문서 길이 편향에 안전하고, (2) K-means가 도메인 수를 아는 상황에서 DBSCAN보다 다루기 쉬우며, (3) TruncatedSVD는 필수는 아니지만 n_components=5 근처로 작게 잡으면 소폭의 안정성 이득이 있다.

확인 필요/미확인 항목

① Count+KMeans 등 SVD 실험에 포함되지 않은 조합의 ARI/NMI 미계산(정규화 전후 비교라는 정황 근거로만 추론) ② 벡터화 행렬(4,610차원)의 완전 중복 컬럼 존재 여부 ③ 군집별(전자제품/호텔/자동차) 감성 분포 정확한 수치는 이미지로 저장되지 않아 미인용.

5 Mercari Price Suggestion

상품명·카테고리·브랜드·설명 텍스트로 중고거래 판매 가격을 예측하는 회귀 문제 — 담당 모델: LinearRegression

5.1 프로젝트 개요

상품의 이름·카테고리·브랜드·상태·배송조건·설명 텍스트로 판매 가격(price)을 예측하는 문제다. price가 0 이상의 연속적인 실수값이므로 회귀 문제로 정의된다. 데이터는 1,482,535행 × 8열이며, 염운호는 LinearRegression을 주 모델로, Ridge/Lasso/RandomForest/XGBoost는 Optuna로 튜닝한 비교용 모델로 사용했다.

5.1.1 입력 피쳐 구성

피쳐명	성격	비고
train_id	식별자	학습 미사용
name	텍스트	상품명, CountVectorizer 벡터화
item_condition_id	범주형(순서형)	1~5등급, 원-핫 인코딩
category_name	범주형	"대/중/소" 계층 텍스트, "/" 분할 필요
brand_name	범주형	결측 다수, 원-핫 인코딩
shipping	범주형(이진)	0/1, 배송비 부담 주체
item_description	텍스트	상품 설명, TF-IDF 벡터화
price	수치형(타겟)	log1p 변환 후 학습

5.1.2 EDA 요약 통계

1,482,535 전체 행 수	\$26.74 price 평균	\$17.00 price 중앙값	\$2,009 price 최댓값
----------------------------	----------------------------	-----------------------------	-----------------------------

price=0인 행이 874건 존재한다(실거래에서 이례적인 값이나 이 노트북에서는 별도 제거 처리는 없음). shipping은 0(819,435건)/1(663,100건)로 비교적 균일하다. item_condition_id는 1등급(640,549건)이 가장 많고 5등급(2,384건)이 가장 적어, 상품 상태가 좋은 쪽에 치우쳐 있다.

5.1.3 결측치 현황

컬럼	결측 건수
category_name	6,327
brand_name	632,682
item_description (진짜 NaN)	6

컬럼	결측 건수
item_description ("No description yet" 포함, 사실상 결측)	82,489

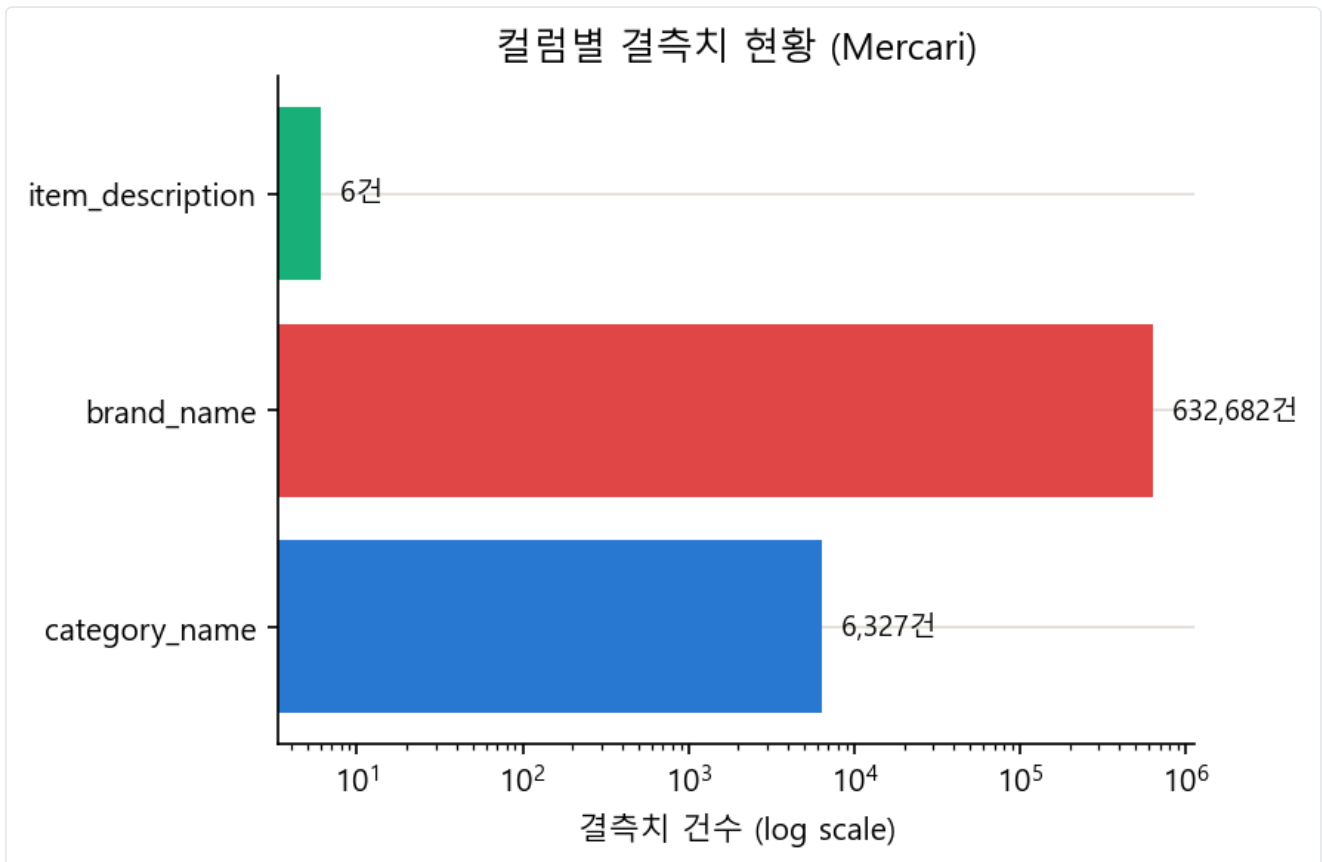


그림 5-1. 컬럼별 결측치 현황(로그 스케일) — brand_name 결측이 압도적으로 많다.

세 컬럼의 결측(사실상 결측 포함)을 모두 합치면 $6,327 + 632,682 + 82,489 + 6 = 721,504$ 건이다. brand_name 결측 비율이 약 43%로 가장 커서, 단순 fillna보다 name 텍스트에서 브랜드를 역추론하는 보완 로직이 필요했다(5.3).

5.1.4 Feature 이름 특징

컬럼명은 모두 snake_case 규칙을 따른다. item_condition_id 는 이름에 "id"가 있지만 실제로는 순서가 있는 등급 범주형이다. category_name 은 단일 카테고리가 아니라 "대분류/중분류/소분류"가 "/"로 결합된 복합 문자열이다. shipping 은 배송비 자체가 아니라 배송비 부담 주체를 나타내는 이진 플래그다.

5.1.5 동일 값인 Feature

원본 8개 컬럼 중 값이 하나로 고정된(분산 0) 컬럼은 없다.

5.1.6 중복 Feature

train_id 포함 전체 8개 컬럼 기준 완전 중복 행은 0건이다(train_id가 모두 고유하므로 당연). train_id를 제외한 나머지 7개 컬럼만으로 보면 49건이 중복이다. 서로 다른 상품(train_id)인데 나머지 속성이 완전히 동일한 경우로, 동일 상품이 중복 등록됐거나 우연히 속성이 일치한 사례로 추정된다. 이 노트북에서는 별도 제거는 하지 않았다.

5.1.7 타겟 변수(price) 분포 분석

원본 price는 왜도 11.39의 매우 심한 오른쪽 꼬리 분포다. `log1p` 변환 후 왜도는 0.66으로 크게 줄어 정규분포에 가까워진다. 로그 변환된 price로 학습한 뒤, 평가 시 `expm1` 으로 원래 가격 스케일로 복원해 RMSLE를 계산한다.

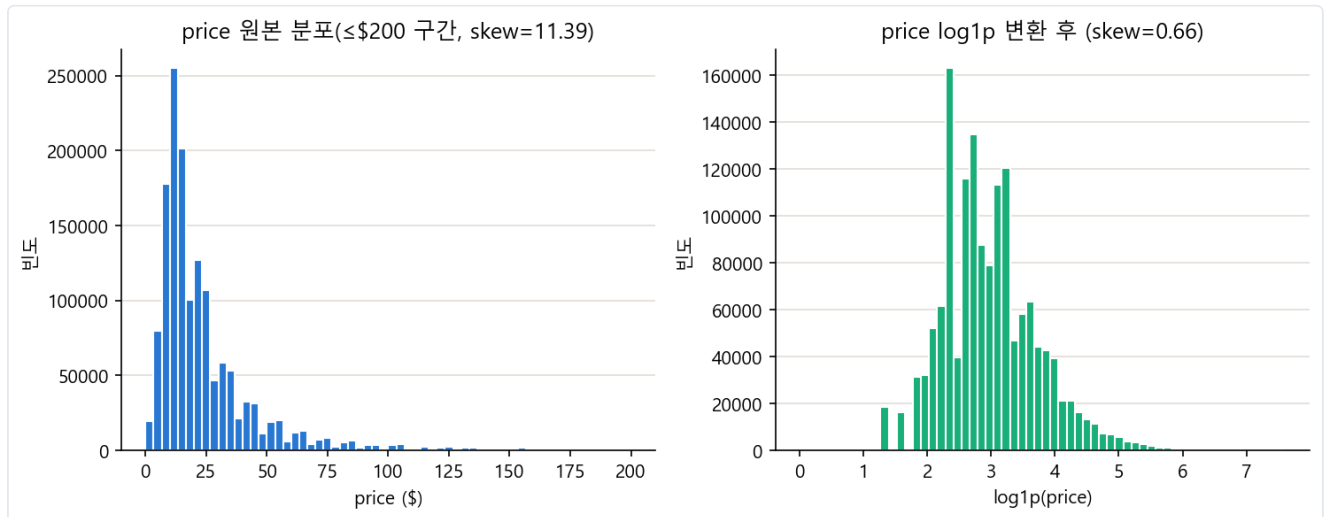


그림 5-2. price 원본 분포(좌, 왜도 11.39)와 log1p 변환 후 분포(우, 왜도 0.66)

5.2 결과 지표

5.2.1 결과 지표 정의 및 선정 근거

RMSLE (Root Mean Squared Logarithmic Error)

$$\text{RMSLE} = \sqrt{\text{mean}((\log(1+\hat{y}) - \log(1+y))^2)}$$

RMSE 대신 RMSLE를 쓰는 이유는 두 가지다. 첫째, price가 오른쪽으로 치우친 분포라 절대오차(RMSE)는 고가 상품 오차에 과도하게 좌우되지만, RMSLE는 로그를 취해 절대오차가 아닌 비율(상대) 오차를 평가한다. 실제 \$10인데 \$15로 예측한 경우와 실제 \$100인데 \$150으로 예측한 경우, 절대오차는 \$5와 \$50으로 다르지만 둘 다 1.5배라는 동일한 비율 오차이며 RMSLE는 이를 비슷하게 평가한다. 둘째, 로그 차 형태이므로 실제보다 낮게 예측했을 때(과소예측)의 벌점이 더 크게 작용하는 비대칭성이 있어, 가격을 지나치게 낮게 부르는 실수를 상대적으로 더 강하게 억제한다.

5.2.2 결과 지표 해석 계획

RMSLE는 로그 스케일 오차이므로 `exp(RMSLE)`를 계산하면 "실제 가격 대비 예측 가격의 평균적인 배율 오차"로 환산할 수 있다. 모델 간 비교는 동일한 피쳐 세트·동일한 train/test 분할(random_state=42, test_size=0.2) 결과만 비교해 공정성을 확보한다. Ridge/Lasso는 회귀계수 크기와 부호로 어떤 피쳐가 가격을 얼마나, 어느 방향으로 움직이는지 해석하는 데 사용한다.

5.3 데이터 전처리 & 클리닝

가격 이상치 제거, 텍스트 정제(소문자화/축약어 복원/특수문자 제거) 단계는 이 노트북에 존재하지 않는다(price=0 874건, 중복 49건에 대한 별도 제거 코드도 없음). 실제 수행된 전처리는 다음 세 가지다.

```
# ① 타깃 로그 변환
mercari_df['price'] = np.log1p(mercari_df['price'])

# ② category_name 계층 분할
def split_cat(category_name):
    try: return category_name.split('/')
    except: return ['Other_Null', 'Other_Null', 'Other_Null']
mercari_df['cat_dae'], mercari_df['cat_jung'], mercari_df['cat_so'] = \
    zip(*mercari_df['category_name'].apply(split_cat))
```

```
# ③ brand_name 결측 보완: name 텍스트에서 브랜드명 역추론
known_brands = set(mercari_df.loc[mercari_df['brand_name'].notnull(), 'brand_name'].unique())
def find_brand_in_name(name, brand_set):
    words = str(name).split()
    for size in (3, 2, 1):
        for i in range(len(words)-size+1):
            cand = ' '.join(words[i:i+size])
            if cand in brand_set: return cand
    return None
```

brand_name 결측(632,682건) 중 148,714건이 name 텍스트 역추론으로 채워졌고, 나머지 483,968건만 최종적으로 'Other_Null'로 채워졌다. 브랜드를 새 카테고리로 만들지 않고 기존 값 중에서만 채우므로, 이후 원-핫 인코딩 시 브랜드 차원 수(4,810개)가 늘어나지 않는다는 것이 이 방식의 핵심 근거다. category_name(6,327건), item_description도 동일하게 fillna('Other_Null')로 처리했다.

5.4 피쳐 구조화 & 엔지니어링

실제로 만들어진 파생 피쳐는 category_name을 분할한 cat_dae/cat_jung/cat_so 3개뿐이며, desc_word_count-name_len 같은 길이 기반 파생 피쳐는 없다. cat_so (소분류, 871개 유형)는 cat_dae/cat_jung과 계층적으로 겹치는 정보가 많고 카디널리티가 커서 선형모델 계수 추정이 불안정해질 수 있다는 이유로 최종 피쳐에서 제외됐다.

피쳐	벡터화 방식	파라미터	결과 차원
name	CountVectorizer	min_df=2, max_features=20000	20,000
item_description	TfidfVectorizer	max_features=20000, ngram(1,3), stop_words='english'	20,000

name은 평균 길이가 짧아 단어 빈도 차이를 구분하기 어려우므로 단순 빈도 기반 CountVectorizer를 쓰고, 저빈도 노이즈 제거를 위해 min_df=2를 적용했다. item_description은 길고 공통 단어가 많아 핵심 단어에 가중치를 주는 TF-IDF를 쓰고, ngram_range=(1,3)으로 "space gray", "gift card" 같은 구 단위 정보까지 포착했다. brand_name/item_condition_id/shipping/cat_dae/cat_jung은 LabelBinarizer로 원-핫 인코딩했다. 최종 피쳐 행렬은 20,000+20,000+4,810+5+1+11+114=44,941차원이다(노트북 실행 결과로 직접 확인).

확인 필요

총 취합 자료에 기재된 "161,569개 컬럼"은 이 LinearRegression_pure 노트북의 최종 피쳐 행렬 크기(44,941차원)와 다르다. 다른 파이프라인이나 이전 버전 노트북 수치일 가능성이 있어 재확인이 필요하다.

5.5 모델링 & 하이퍼파라미터 튜닝

LinearRegression은 정규화 항이 없어 튜닝할 하이퍼파라미터가 사실상 없다. 대신 "피쳐 조합" 차원에서 비교했다. item_description 포함 시 RMSLE 0.4818, 제외 시 0.5154로, 고차원 텍스트 피쳐 포함이 성능을 개선시켰다.

비교용 모델은 먼저 수동 그리드로 대략적 경향을 확인한 뒤 Optuna로 연속 구간을 재탐색했다(20만 행 샘플로 탐색 후 최적값으로 전체 148만 행 재학습).

모델	Optuna 탐색 구간	트라이얼
Ridge	$\alpha \in [1e-3, 1e2]$ (log)	15
Lasso	$\alpha \in [1e-4, 1e-1]$ (log)	15
RandomForest	n_estimators[50,150], max_depth[6,14], min_samples_leaf[1,5]	6
XGBoost	n_estimators[100,400], max_depth[3,10], lr[0.01,0.3](log) 등	15

Lasso는 수동 비교에서 $\alpha=0.1$ 이상이 모두 RMSLE 0.7511로 수렴(계수가 전부 0이 되어 절편만 남음)해, 탐색 구간을 더 낮은 $[1e-4, 1e-1]$ 로 좁혔다. RandomForest는 44,941차원 희소 행렬에서 트라이얼 비용이 크므로 6회로 제한했다. XGBoost는 `tree_method='hist'`를 써도 44,941열 전체를 148만 행에 그대로 넣으면 메모리 할당에 실패해, 40만 행으로 서브샘플링해 학습했다.

5.6 성능 검증 결과

5.6.1 파이프라인 요약 다이어그램



5.6.2 지표 결과

염윤호가 채택한 LinearRegression의 RMSLE는 **0.4818**(item_description 포함 피쳐 세트)이다.

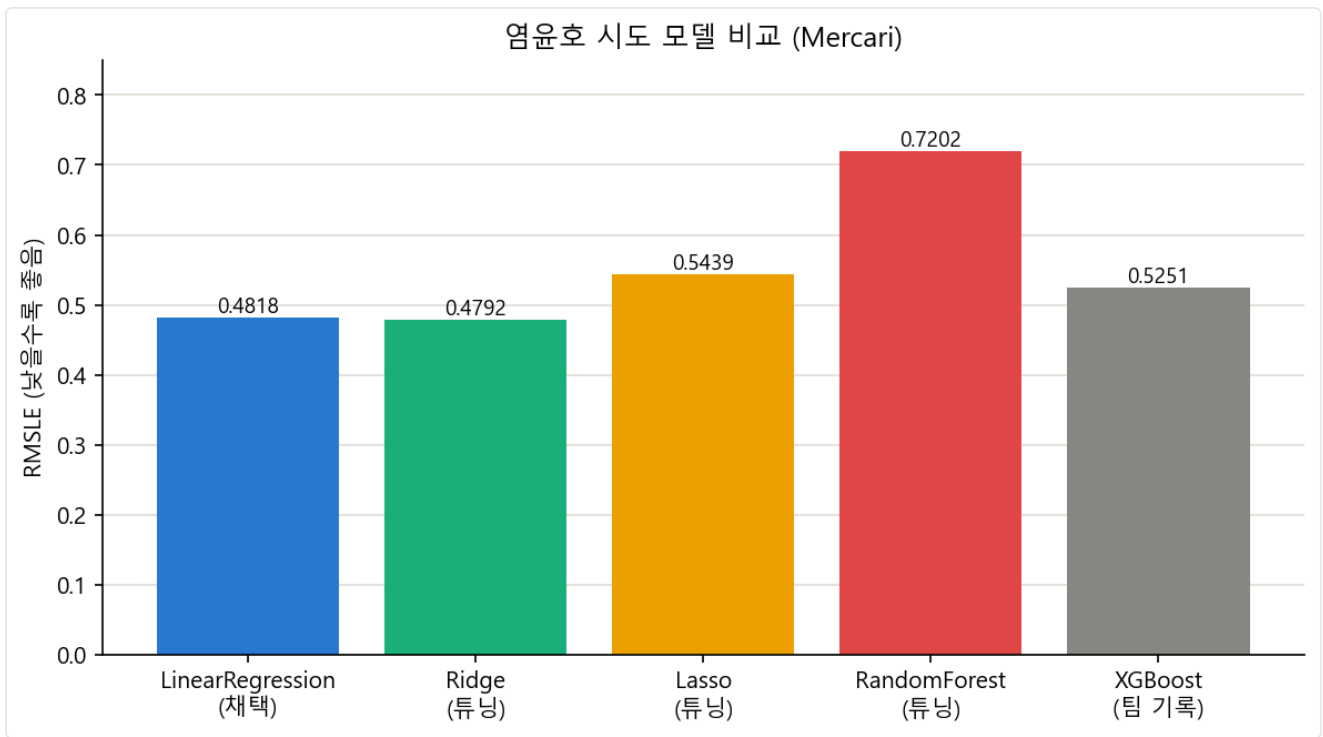


그림 5-3. 염운호가 시도한 5개 모델의 RMSLE 비교(낮을수록 좋음)

5.6.3 지표 해석

RMSLE 0.4818을 $\exp(0.4818) \approx 1.62$ 배율로 환산하면, 예측값이 실제 가격 대비 대략 1.62배 범위 안에서 형성된다고 해석할 수 있다. 예를 들어 실제 가격이 \$50인 상품이라면 예측 가격은 대략 $\$50/1.62 \approx \$31 \sim \$50 \times 1.62 \approx \81 범위 정도의 오차 수준으로 형성된다(근사적인 배율 환산이며 엄밀한 신뢰구간은 아니다). 실제 가격이 \$10인 저가 상품이면 같은 배율로 약 \$6.2~\$16.2, \$100인 고가 상품이면 약 \$62~\$162 범위로, 저가 상품일수록 절대 금액 오차는 작지만 상대적 체감 오차(비율)는 동일하게 유지된다는 것이 RMSLE 해석의 핵심이다. item_description을 제외하면 RMSLE가 0.5154($\exp \approx 1.67$)로 악화되는데, 이는 상품 설명 텍스트에 가격을 결정하는 유의미한 정보가 많이 담겨 있음을 뜻한다.

5.6.4 모델별 결과 비교 및 추론

모델	RMSLE	비고
LinearRegression(채택)	0.4818	-
Ridge	0.4792	Optuna 튜닝(alpha≈5.59)
Lasso	0.5439	Optuna 튜닝(alpha≈0.0001)
RandomForest	0.7202	Optuna 튜닝(20만 행 탐색 → 전체 재학습)
XGBoost	0.5251	팀 취합 기록치(이 노트북 파일에서는 출력 미저장, 미확인)

확인 필요 — 튜닝 전/후 라벨

과제 지시문의 "Lasso 0.6923(Optuna 튜닝)", "RandomForest 0.7063(Optuna 튜닝)"은 실제로는 **튜닝 전(수동 설정)** 수치로 확인됐다. Optuna 튜닝 후 재학습 결과는 각각 Lasso 0.5439, RandomForest 0.7202(오히려 악화)이며, 본 절의 표는 노트북 실행 결과 기준(튜닝 후)으로 표기했다. XGBoost 0.5251은 이 노트북 파일 안에서는 출력이 저장되어 있지 않아 팀 취합 자료 값을 인용했다.

Ridge가 LinearRegression보다 근소하게 나은 이유: 정규화가 없는 LinearRegression은 44,941차원 희소 고차원 피처에서 다중공선성(name과 item_description의 겹치는 단어, cat_dae/cat_jung의 계층적 상관 등)에 취약해 계수가 불안정해질 수 있다. Ridge는 L2 정규화로 계수 크기를 완만히 수축시켜 분산을 줄이므로 아주 소폭 개선된다. 다만 그 차이(0.4818→0.4792)는 작아, 이 피처 세트에서 정규화 효과가 제한적이라는 뜻이기도 하다.

Lasso가 더 나쁜 이유: L1 정규화는 계수를 정확히 0으로 만들어 피처를 제거하는데, alpha=0.1 이상에서는 계수가 모두 0이 되어(RMSLE 0.7511) 절편만 남는다. Optuna로 더 정밀 탐색해도 최적 alpha가 0.0001 근처로 극히 작게 나왔다(RMSLE 0.5439). 이는 44,941차원 대부분이 저빈도·약한 신호이기 때문에, 조금이라도 유용한 피처가 남으려면 정규화 강도를 거의 0에 가깝게 낮춰야 한다는 뜻이다. 즉 L1의 공격적인 피처 선택이 오히려 유용한 정보를 과도하게 깎아낸 것으로 추론된다.

RandomForest-XGBoost가 선형모델보다 나쁜 이유: 선형모델은 20,000+20,000+4,810개의 희소한 원-핫/TF-IDF 차원 각각에 독립적으로 가중치를 부여할 수 있다. 반면 트리 모델은 분할 시 한 번에 하나의 피처만 기준으로 나누는 구조라, 극희소(대부분 값이 0)한 44,941차원에서는 유의미한 분할 기준을 찾기 어렵다. RandomForest는 `max_features='sqrt'` 로 후보 피처 수까지 줄여 정보 손실이 더 크다. 또한 Optuna 튜닝 RandomForest (0.7202)가 튜닝 전(0.7063)보다 오히려 나빠졌는데, 20만 행 샘플에서 찾은 최적 하이퍼파라미터가 148만 행 전체 재학습에서는 최적이지 않았을 가능성을 보여준다. XGBoost(0.5251)는 부스팅 구조상 RandomForest보다는 낫지만 여전히 선형모델보다 뒤처지며, 같은 논리(희소 고차원에서 트리 분할보다 선형 가중치가 유리)로 설명된다.

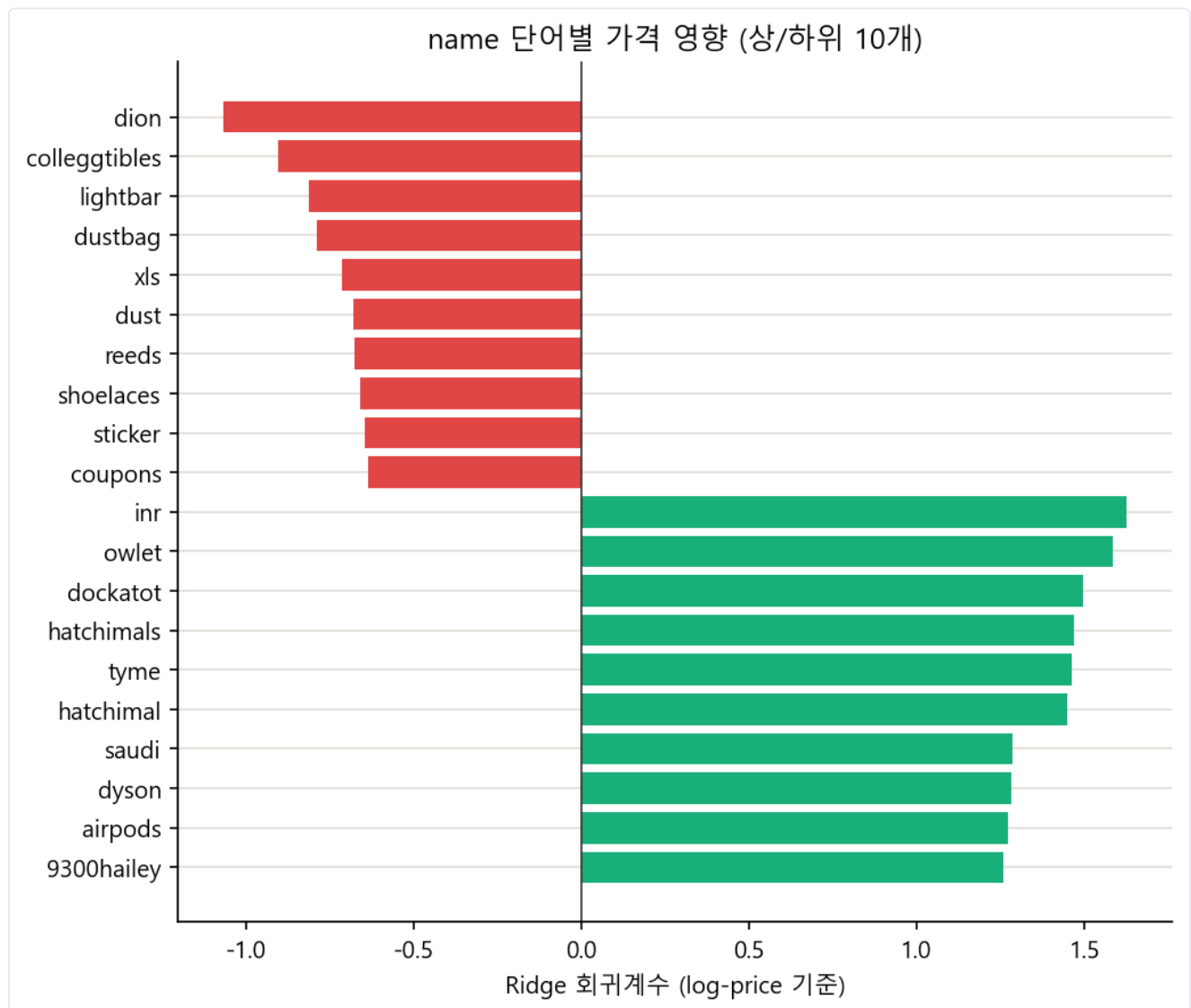


그림 5-4. name 단어별 Ridge 회귀계수 상/하위. "inr", "owlet", "dockatot" 등 고가 브랜드/유아용품 관련 단어가 가격을 밀어올리고, "dion", "colleggtibles" 등은 낮춘다.

계수 시각화는 선형모델의 해석력을 잘 보여준다. 브랜드 계수는 David Yurman·Tiffany & Co·Christian Louboutin 같은 명품 브랜드가 큰 양(+)의 계수를, eos·Field & Stream 같은 저가 생활용품 브랜드가 음(-)의 계수를 가져 브랜드 프리미엄이 선형적으로 잘 포착된다. 대분류 계수는 Electronics(+0.15)·Men(+0.10)이 가격을 밀어올리고 Beauty(-0.12)·Home(-0.10)이 낮추는 방향으로 작용해 카테고리별 가격대 차이가 직관적으로 해석된다. item_condition_id 계수는 1등급(+0.23)에서 5등급(-0.28)까지 단조적으로 감소해 "숫자가 클수록 상태가 나쁘다"는 도메인 지식과 일치한다. shipping 계수(-0.19)는 판매자가 배송비를 부담하는 경우 상품 가격 자체가 낮게 책정되는 경향(배송비만큼 가격에서 할인해 부담을 상쇄)을 보여준다. 이처럼 선형모델은 각 피처의 영향을 계수 하나로 직접 확인할 수 있어, 트리 모델보다 결과 해석과 도메인 검증이 용이하다는 장점이 이 문제에서도 드러난다.

확인 필요/미확인 항목

① XGBoost RMSLE(0.5251)는 이 노트북 파일 내 실행 출력으로 재검증 불가(팀 취합 자료 인용) ② 최종 피처 컬럼 수 44,941 vs 총 취합 자료의 161,569 불일치 ③ price=0(874건)·중복행(49건)에 대한 별도 정제 미실시.

6 회고

공백 — 추후 추가 예정

본 절은 프로젝트 종료 후 팀 회고 내용을 반영하기 위해 비워둔다.

7 참고문헌

공백 — 추후 추가 예정

본 절은 인용 문헌 및 참고 자료 목록을 추후 반영하기 위해 비워둔다.